



Bocskai István Református Oktatási Központ

Mészáros Bálint Péter, Haas Dániel, Wágner Ferenc Mihály

13.C

Szoftverfejlesztő és -tesztelő

(2025-2026)

Tartalom

1. Bevezetés.....	9
1.1 Röviden a projektről.....	9
1.2 Témaválasztás indoklása.....	9
1.3 Célkitűzések.....	9
2. A főoldal.....	10
2.1 Funkcionális leírás	10
2.1.1 Főbb funkciók:.....	10
2.3 Frontend felépítés (HTML és CSS)	10
2.3.1 Szerkezeti felépítés (index.html)	10
2.3.2 Megjelenés és reszponzivitás (index.css, style.css).....	11
2.3 Kliensoldali logika (JavaScript).....	11
2.3.1 Felhasználói interakciók kezelése.....	11
2.3.2. Végtelenített Slider folyamat.....	11
2.3.3. Dinamikus ajánló modul	12
2.3.4 Hibakezelés és visszajelzés.....	12
2.4 Tesztelési napló.....	12
3. Regisztráció.....	14
3.1 Funkcionális leírás.....	14
3.1.1 Főbb funkciók:	14
3.2 Frontend felépítés (HTML és CSS).....	14
3.2.1 Szerkezeti felépítés (signUp.html)	14
3.2.2 Megjelenés és reszponzivitás (signIn.css, style.css).....	15
3.3 Programozási logika (signUp.js).....	15
3.3.1 Felhasználói interakciók kezelése	15
3.3.2 Regisztrációs folyamat	16
3.3.3 Google alapú regisztráció	17
3.4. Hibakezelés	18
3.5 Tesztelési napló	18
4. Bejelentkezés.....	21
4.1 Funkcionális leírás.....	21
4.1.1 Főbb funkciók:	21
4.2 Frontend felépítés (HTML és CSS).....	21
4.2.1 Felületi elemek (signIn.html)	21

4.2.2 Design és reszponzivitás (signIn.css)	22
4.3 Programozási logika (signIn.js)	22
4.3.1 Jelszó láthatóságának kezelése.....	22
4.3.2 E-mail-jelszó alapú hitelesítés	22
4.3.3 E-mail megerősítés ellenőrzése	23
4.3.4 Google alapú belépés	23
4.3.5 Megerősítő e-mail újraküldése	23
4.4 Hibakezelés	24
4.5 Tesztelési napló	24
4.6 Programozási logika (signIn.js)	26
4.6.2 Funkcionális leírás	26
4.6.3 Főbb funkciók:.....	26
5. Profil	28
5.1 Funkcionális leírás	28
5.1.1 Főbb funkciók:.....	28
5.2 Frontend felépítés (HTML és CSS)	28
5.2.1 Felületi elemek (profil.html)	28
5.2.2 Design és megjelenés (profil.css)	28
5.3 Programozási logika (profil.js)	29
5.3.1 Hozzáférés-ellenőrzés	29
5.3.2 Profiladatok betöltése	29
5.3.3 Foglalások kezelése	29
5.3.4 Kijelentkezési folyamat	29
5.4 Tesztelési napló.....	30
6. Shop	31
6.1 Funkcionális leírás	31
6.1.1 Főbb funkciók:.....	31
6.2 Frontend felépítés (HTML és CSS)	31
6.2.1 Felületi struktúra (shop.html)	31
6.2.2 Design és UX (User Experience)(shop.css).....	32
6.3 Programozási logika (shop.js	32
6.3.1 Termékadatok kezelése.....	32
6.3.2 Keresés, szűrés és rendezés	32
6.3.3 Termékmodál működése.....	33
6.3.4 Kedvencek és kosár kezelése.....	33
6.3.5 Reszponzív szűrőlogika	33

6.4 Tesztelési napló.....	34
7. Kedvencek	36
7.1 Funkcionális leírás	36
7.1.1 Főbb funkciók:.....	36
7.2 Frontend felépítés (HTML és CSS)	36
7.2.1 Felületi szerkezet.....	36
7.2.2 Design és megjelenés (favourite.css)	36
7.3 Programozási logika (favourite.js)	37
7.3.1 Betöltés és megjelenítés	37
7.3.2 Kosárba helyezés kedvencekből.....	37
7.3.3 Törlés a kedvencek közül	37
7.4 Tesztelési napló.....	38
8. Kosár	40
8.1 Funkcionális leírás.....	40
8.1.1 Főbb funkciók:	40
8.2 Frontend felépítés (HTML és CSS).....	40
8.2.1 Felületi struktúra (cart.html).....	40
8.2.2 Design és megjelenés (cart.css).....	40
8.3 Programozási logika (cart.js)	41
8.3.1 Betöltés és szinkronizáció	41
8.3.2 Dinamikus renderelés.....	41
8.4 Tesztelési napló	42
10. Edzéstervék és Gyakorlatok	48
10.1 Funkcionális leírás	48
10.1.1 Főbb funkciók:.....	48
10.2.1 Szerkezeti felépítés (workouts.html)	49
10.3 Programozási logika (workouts.js).....	50
10.3.1 Modális vezérlés és Adatkezelés.....	50
10.3.2 Kosár szinkronizáció.....	50
10.4 Tesztelési napló.....	51
11. Edzőtermek.....	54
11.1 Funkcionális leírás	54
11.1 Főbb funkciók:.....	54
11.2 Frontend felépítés (HTML és CSS)	54
11.2.1 Szerkezeti felépítés (gym.html és aloldalai)	54
11.2.2 Megjelenés és reszponzivitás (gym.css, style.css).....	55

11.3 Kliensoldali logika (JavaScript).....	55
11.3.1. Felhasználói interakciók kezelése.....	55
11.3.2 Galéria vezérlés (gym.js).....	56
11.4 Kontakt és Külső Integrációk	56
4. Tesztelési napló.....	56
1. Teszteset: Horgony-navigáció ellenőrzése.....	56
2. Teszteset: Galéria léptetés	57
3. Teszteset: Kontakt hivatkozások	57
12. Edzői Rendszer (Trainers & Profiles)	58
12.1 Funkcionális leírás	58
12.1.1 Főbb funkciók:.....	58
12.2 Frontend felépítés (HTML és CSS)	59
12.2.1 Gyűjtőoldal és Profil szerkezet.....	59
12.2.2 Vizuális stílusjegyek (trainers.css).....	59
12.3 Programozási logika és Navigáció.....	60
12.3.1 Dinamikus útvonalválasztás	60
12.4 Tesztelési napló (Hayoto profilja alapján modellezve).....	60
13. Időpontfoglalás.....	62
13.1 Funkcionális leírás	62
13.1.1 Főbb funkciók:.....	62
13.2 Frontend felépítés (HTML és CSS)	63
13.2.1 Szerkezeti felépítés (booking.html)	63
13.2.2 Megjelenés és responszivitás (booking.css).....	64
13.3 Programozási logika (booking.js).....	64
13.3.1 Dinamikus tartalomkezelés.....	64
13.3.2 Egyedi naptárlogika.....	65
13.3.3 Felhasználói adatok integrációja.....	66
13.3.4 Foglalási folyamat és mentés	66
14. Anatómia oldal.....	68
14.1 Funkcionális leírás	68
14.1.1 Főbb funkciók:.....	68
14.2 Frontend felépítés (HTML és CSS)	68
14.2.1 Szerkezeti felépítés (anatomy.html)	68
14.2.2 Megjelenés és responszivitás (anatomy.css, style.css)	69
14.3 Kliensoldali logika (JavaScript).....	69
14.3.1 Felhasználói interakciók kezelése.....	69

14.3.2 Dinamikus tartalomfrissítés.....	69
14.3.3 iFrame konfiguráció	70
14.3.4 Hibakezelés.....	70
4. Tesztelési napló.....	70
15. BMI Kalkulátor	72
15.1 Funkcionális leírás	72
15.1.1 Főbb funkciók:.....	72
15.2 Frontend felépítés (HTML és CSS)	73
15.2.1 Szerkezeti felépítés (calculatorBMI.html).....	73
15.3 Programozási logika (calculatorBmi.js)	74
15.3.1. Számítási algoritmus.....	74
15.3.2 Dinamikus UI frissítés (updateUI függvény)	74
15.3.3 SVG Animáció technológiája.....	75
15.4 Tesztelési napló.....	76
2. Teszteset: Színváltás és extrém értékek.....	76
3. Teszteset: Üres mezők kezelése	76
16. Elérhetőségek.....	77
16.1 Funkcionális leírás	77
16.1.1 Főbb funkciók:.....	77
16.2 Frontend felépítés (HTML és CSS)	78
16.2.1 Szerkezeti felépítés (contacts.html)	78
16.2.2 Megjelenés és reszponzivitás (contacts.css)	78
16.3 Programozási logika (contact-sender.js).....	79
16.3.1 EmailJS Integráció	79
16.3.2 Állapotkezelés és UX.....	80
16.3.3 Egyedi Modális visszajelzés (showStatusModal)	80
16.4 Tesztelési napló.....	81
1. Teszteset: Sikeres üzenetküldés.....	81
2. Teszteset: Kötelező mezők ellenőrzése	81
3. Teszteset: E-mail formátum validáció.....	82
4. Teszteset: Térkép hover effektus	82
17. Vásárlói vélemények	83
17.1 Funkcionális leírás	83
17.1.1 Főbb képességek:	83
17.2 Frontend felépítés és Megjelenés.....	83
18.2.1 Design alapelvek (reviews.css)	83

17.2.2 Reszponzivitás.....	84
17.3 Technikai megvalósítás (Backend & Logic)	84
17.3.1 Adatbázis integráció (review-handler.js)	84
17.3.2 Logikai számítások és algoritmusok.....	85
17.3.3 Hibaüzenetek és Felhasználói Visszajelzések.....	86
17.4 Tesztelési napló.....	87
1. Teszteset: Jogosultságok és hozzáférés.....	87
2. Teszteset: Új vélemény beküldése.....	87
3. Teszteset: Törlési funkció védelme	88
4. Teszteset: Adatbázis szinkron.....	88
18. Gyakran Ismételt Kérdések.....	89
18.1 Funkcionális leírás	89
18.1.1 Főbb jellemzők:	89
18.2 Frontend felépítés és Megjelenés.....	89
18.2.1 Design alapelvek (questions.css).....	89
18.2.2 Az animáció technikai megvalósítása.....	90
.....	90
18.3 Technikai megvalósítás (Backend & Logic)	91
18.3.1 JavaScript vezérlés (questions.html belső script)	91
18.3.2 Navigációs logika	91
18.4 Tesztelési napló.....	92
1. Teszteset: Harmonika működése	92
2. Teszteset: Többszörös nyitás.....	92
3. Teszteset: Reszponzivitás	93
.....	93
4. Teszteset: Fordító modul	93
19. Süti beállítások	94
19.1 Funkcionális leírás	94
19.1.1 Főbb képességek:	94
19.2 Frontend felépítés és Megjelenés.....	94
20.2.1 Design alapelvek (cookies.css)	94
19.2.2 Vizuális jelzések	95
19.3. Technikai megvalósítás (Backend & Logic)	95
19.3.1 Adatkezelés (cookies.js)	95
19.3.2 UI Logika és Animációk	96
19.4 Tesztelési napló.....	96

1. Teszteset: Alapértelmezett állapot	96
2. Teszteset: Beállítások mentése	96
3. Teszteset: Adatperzisztencia.....	97
20 Közösségi média integráció és marketing szerepe	98
20.1 Alkalmazott megoldások:	98
20.2 Tesztelési napló (Közösségi média kiegészítés).....	99
1 Teszteset: Külső hivatkozások működése	99
2 .Teszteset: Közösségi média ikonok interakciója.....	99
21 Adatbázis	100
21.1 firebase.js – Infrastruktúra és konfiguráció.....	100
21.1.1 A modul főbb feladatai:	100
21.1.2 A modulhoz kapcsolódó fontosabb adatbázis-gyűjtemények:	101
21.2 utils.js – Segédfüggvények és UI Komponensek	103
21.2.1 A modul főbb jellemzői:	103
21.2.2 Kódmagyarázat:.....	104
21.3 auth-header.js – Dinamikus Navigáció és Állapotkezelés	105
21.3.1 A modul főbb feladatai:	105
21.3.2 Kódmagyarázat:.....	106
21.4 Tesztelési napló.....	107

1. Bevezetés

1.1 Röviden a projektről

A ForgeX vizsgaremek egy olyan komplex webes platform, amelyet a sportolni vágyók és az egészségtudatos életmódot követők számára fejlesztettük ki. Az oldal célja, hogy egyetlen felületen kínáljon minden olyan eszközt és információt, amely a fizikai fejlődéshez és az egészséges életvitel kialakításához szükséges. A projekt sokrétűségét a különböző funkciók adják: a felhasználók a BMI-kalkulátor segítségével pontos képet kaphatnak aktuális testi állapotukról, majd az interaktív Gyakorlatok és Anatómia menüpontok révén szakmai alapokra helyezhetik edzéseiket. Az oldal lehetőséget biztosít személyi edzőkhöz történő időpontfoglalásra, a Webshopon keresztül pedig prémium sportruházat és táplálékkiegészítők beszerzésére is mód nyílik. A ForgeX nem csupán egy információs portál, hanem egy interaktív segítőtárs, amely a méréstől a gyakorlati megvalósításon át a felszerelés biztosításáig támogatja a felhasználót céljai elérésében.

1.2 Témaválasztás indoklása

Választásunk azért esett a sport és az egészség témakörére, mert csapatunk tagjai számára a mindennapi mozgás és az egészségmegőrzés kiemelt jelentőséggel bír. A projekt során saját érdeklődésünket és elkötelezettségünket kívántuk szakmai keretek közé emelni.

1.3 Célkitűzések

Projektünk fő célkitűzése egy olyan komplex webes platform kidolgozása volt, amely minden sportot kedvelő felhasználó számára teljes körű támogatást nyújt a fejlődéshez. Az oldal integrált szolgáltatásaival – legyen szó edzőterem-hálózatról, személyi edzésről vagy speciális gyakorlatcsomagokról – törekedtünk arra, hogy kortól és edzettségi szinttől függetlenül bárki megtalálja a céljaihoz szükséges eszközöket és szakmai háttérrel.

2. A főoldal

2.1 Funkcionális leírás

A főoldal célja a ForgeX márka bemutatása, a felhasználók vizuális elkötelezése, valamint a platform különböző szolgáltatásai (Shop, Edzők, Anatómia, Kalkulátorok) közötti központi elosztó felület biztosítása. Az oldal dinamikus elemekkel és modern vizuális megoldásokkal ösztönzi a látogatókat az interakcióra.

2.1.1 Főbb funkciók:

- Reszponzív navigációs rendszer (asztali és mobil nézet)
- Interaktív, végtelenített termék/szolgáltatás slider
- Dinamikus tartalomváltó szekció (ForgeX gépek bemutatása)
- Anatómiai kedvcsináló videós háttérelemekkel
- Többnyelvűség támogatása (Magyar, Angol, Német) Google Translate integrációval
- Információs kártyák modális ablakos részletezéssel
- Kosár-számláló valós idejű megjelenítés

2.3 Frontend felépítés (HTML és CSS)

2.3.1 Szerkezeti felépítés (index.html)

A főoldal szemantikus HTML5 struktúrát követ. A fejléc (top-bar) tartalmazza a navigációt és a gyorselérési ikonokat (profil, kosár, kedvencek). A tartalom logikai szekciókra tagolódik: a Hero rész (container-runner) a fő üzenetet közvetíti, ezt követi a szolgáltatások interaktív választója, majd a vizuális slider. Az oldal alsó részén kapott helyet az anatómiai blokk és a hitvallást bemutató Arnold-szekció.

2.3.2 Megjelenés és reszponzivitás (index.css, style.css)

- A design a "Dark Mode" esztétikát követi, domináns fekete háttérrel és a ForgeX egyedi zöld (#00ca65) kiemelő színével. Az oldalon megjelenik a Glassmorphism (üveghatás) a modális ablakoknál.
- A reszponzivitás két szinten valósul meg:
- Asztali nézet: Széles elrendezés, fix felső menüsorral.
- Mobil nézet: A felső ikonok elrejtésre kerülnek, helyüket egy könnyen kezelhető alsó menüsor (phone-menu) és egy oldalról behúzható hamburger-menü veszi át. A nagy képek helyett optimalizált, álló formátumú
- (mobileImage) erőforrások töltődnek be.

2.3 Kliensoldali logika (JavaScript)

2.3.1 Felhasználói interakciók kezelése

A rendszer figyeli a görgetési eseményeket, és a fadeIn osztály segítségével animálva jeleníti meg az elemeket, amikor azok a látótérbe kerülnek. A navigációban a mobil nézetű menü lenyitását és bezárását egy láthatatlan checkbox logikája vezérli,

biztosítva a sima átmeneteket.

2.3.2. Végtelenített Slider folyamat

- Az initSlider funkció felel a képek léptetéséért. A szoftver az első és utolsó kép klónozásával hozza létre a folyamatosság illúzióját.
- Támogatja az automatikus lejátszást (5 másodperces késleltetés).
- Kezeli az egérrel való húzást (drag) és a mobil eszközökön használt érintést (swipe).
- Kattintáskor különbséget tesz a húzás és a navigációs szándék között a data-link attribútum segítségével.

2.3.3. Dinamikus ajánló modul

A gépek bemutatásánál a rendszer egy JavaScript objektumból (recommendationsData) nyeri az adatokat. Kattintáskor a DOM-ban található szövegek és képek forrása anélkül frissül, hogy az oldal újra töltsdne. A script figyeli a képernyő szélességét is, és automatikusan a megfelelő (mobil vagy desktop) képfájlt tölti be a választott géphez.

2.3.4 Hibakezelés és visszajelzés

Az információs kártyák (Eat Clean, Work Hard, Sleep Well) kattintásakor a modul ellenőrzi a data-modal azonosítót. Amennyiben az adat elérhető, az infoModal jelenik meg a tartalommal. A bezárás gomb mellett a rendszer kezeli a modalon kívüli kattintást is, mint bezárási szándékot, javítva ezzel a felhasználói élményt.

2.4 Tesztelési napló

1. Teszteset: Navigáció reszponzivitása

- Bemenet: Ablakméret csökkentése 800px alá.
- Elvart eredmény: A felső menüsor eltűnik, megjelenik a hamburger ikon és az alsó mobil navigáció.
- Eredmény: **Megfelelt**

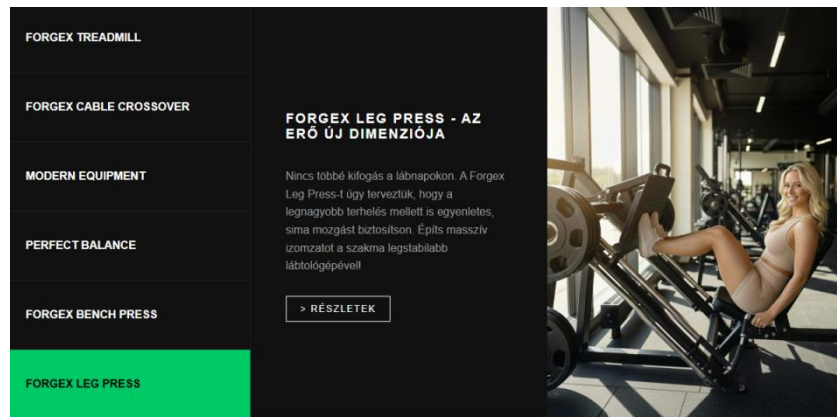
2. Teszteset: Slider végtelenítése

- Bemenet: Az utolsó (4.) slide utáni továbbléptetés.
- Elvart eredmény: A slider észrevehető ugrás nélkül az 1. képre vált.
- Eredmény: **Megfelelt**



3. Teszteset: Dinamikus gépválasztó

- Bemenet: Kattintás a "FORGEX LEG PRESS" menüpontra.
- Elvárt eredmény: A kép, a cím és a leírás azonnal a lábtológép adataira frissül.
- Eredmény: **Megfelelt**



4. Teszteset: Többnyelvűség

- Bemenet: "English" gomb megnyomása.
- Elvárt eredmény: A Google Translate modul aktiválódik, az oldal tartalma angolra vált, a beállítás elmentődik a localStorage-ba.
- Eredmény: **Megfelelt**

3. Regisztráció

3.1 Funkcionális leírás

A regisztrációs modul célja új felhasználók biztonságos létrehozása a ForgeX rendszerben. A felület lehetővé teszi hagyományos e-mail-jelszó alapú regisztrációt, valamint Google-fiókkal történő regisztráció használatát is.

3.1.1 Főbb funkciók:

- teljes név, e-mail cím, telefonszám és jelszó megadása
- a jelszó és a jelszó megerősítése mezők egyezőségének ellenőrzése
- jelszó láthatóságának ki- és bekapcsolása
- Firebase Authentication alapú felhasználó-létrehozás
- kiegészítő felhasználói adatok mentése a Firestore adatbázis users gyűjteményébe
- megerősítő e-mail küldése a regisztráció után
- Google alapú regisztráció támogatása
- hibák kezelése egyedi modális visszajelző ablakkal

3.2 Frontend felépítés (HTML és CSS)

3.2.1 Szerkezeti felépítés (signUp.html)

A regisztrációs oldal egy központi űrlap köré épül, amely logikailag elkülönített mezőket tartalmaz a személyes adatok és a hitelesítési adatok számára. A felhasználó teljes nevet, e-mail címet, telefonszámot, jelszót és jelszó megerősítést ad meg. Az input mezők mellett ikonok segítik az értelmezést, az oldal alján pedig alternatív belépési lehetőségként Google gomb is található.

3.2.2 Megjelenés és reszponzivitás (signIn.css, style.css)

A regisztrációs oldal az alkalmazás többi hitelesítési felületével egységes arculatot használ. A megjelenés alapja az áttetsző, üveghatású kártya, amely háttérrelmosással és finom szegéllyel emelkedik ki a háttérből. A reszponzív kialakításnak köszönhetően mobil eszközön is kényelmesen használható, a mezők és gombok mérete igazodik a kisebb kijelzőhöz.

3.3 Programozási logika (signUp.js)

3.3.1 Felhasználói interakciók kezelése

A jelszómezőknél külön ikonokkal lehet szabályozni, hogy a felhasználó látható vagy rejtett formában szeretné-e megtekinteni a jelszót. A rendszer valós időben ellenőrzi azt is, hogy a két megadott jelszó megegyezik-e, és szükség esetén hibajelzést ad.

```
document.addEventListener( type: 'DOMContentLoaded', listener: () :void => {
  const toggleButtons :NodeListOf<Element> = document.querySelectorAll( selectors: '.toggle-password');

  toggleButtons.forEach( callbackfn: (button :Element ) :void => {
    button.style.cursor = 'pointer';
    button.addEventListener( type: 'click', listener: function () :void {
      const inputField :HTMLInputElement = this.parentElement.querySelector( selectors: 'input');
      if (inputField.type === 'password') {
        inputField.type = 'text';
        this.classList.replace( token: 'fa-eye', newToken: 'fa-eye-slash');
      } else {
        inputField.type = 'password';
        this.classList.replace( token: 'fa-eye-slash', newToken: 'fa-eye');
      }
    });
  });
});
```

3.3.2 Regisztrációs folyamat

Az űrlap elküldésekor a rendszer megakadályozza az oldal újratöltését, majd kliensoldali ellenőrzések után létrehozza a felhasználót a Firebase Authentication szolgáltatásban. Sikeres regisztráció után:

```
try {
  const userCredential = await createUserWithEmailAndPassword(auth, email, password);
  const user = userCredential.user;

  await sendVerificationEmail(user);

  try {
    await savePasswordUserProfile(user, { profile: {
      fullname: fullName,
      phone: phoneNumber,
      email
    }});
  } catch (profileError) {
    console.error("Password profile sync failed:", profileError);
  }

  await syncUserVerificationStatus(user);
  await signOut(auth);|
```

- Megerősítő e-mail kerül kiküldésre
- A felhasználó adatai bekerülnek a Firestore users gyűjteményébe
- A rendszer rögzíti, hogy a fiók e-mailes hitelesítéssel jött létre
- A fiók e-mail megerősítési állapota is mentésre kerül
- A felhasználó kijelentkeztetés után a bejelentkezési oldalra kerül vissza

3.3.3 Google alapú regisztráció

A modul támogatja a Google-fiókkal történő regisztrációt is. Ebben az esetben a Firebase popup alapú hitelesítést használ, majd a felhasználó alapadatai szintén mentésre kerülnek a Firestore adatbázisba. Ha a felhasználó először használja ezt a belépési módot, a rendszer visszajelző üzenetet jelenít meg a sikeres regisztrációról.

```
async function handleGoogleSignup() : Promise<void> { Show usages Balint67
  const provider = new GoogleAuthProvider();
  provider.setCustomParameters({ prompt: 'select_account' });

  setGoogleButtonLoading( {isLoading: true});

  try {
    const result = await signInWithPopup(auth, provider);
    await finalizeGoogleSignup(result);
  } catch (error) {
    console.error("Google Auth Error:", error.code);

    if (error.code === 'auth/popup-blocked') {
      await signInWithRedirect(auth, provider);
      return;
    }
  }
}
```

7

3.4. Hibakezelés

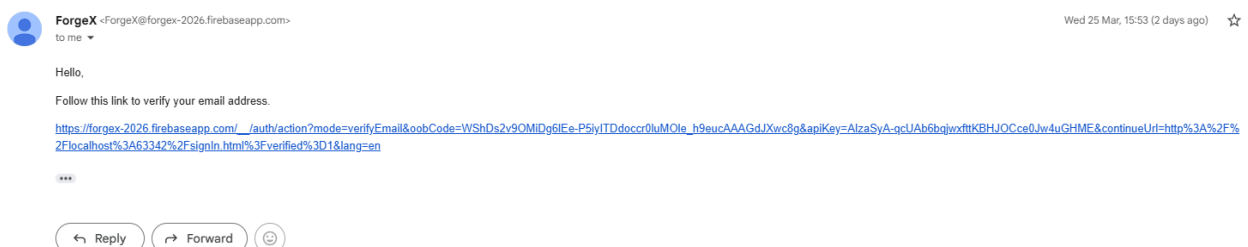
- A regisztráció teljes folyamata try-catch blokkban fut. A rendszer külön kezeli például:
- Az e-mail cím már foglalt
- A jelszó túl gyenge
- A két jelszó nem egyezik meg
- Általános hitelesítési hiba történt

A hibák nem a böngésző alapértelmezett üzeneteivel jelennek meg, hanem a forgeXModal segítségével, egységes felhasználói felületen. A személyreszabott üzenet a tesztelési napló 3. pontjában képpel is illusztráltam.

3.5 Tesztelési napló

1. Teszteset: sikeres regisztráció

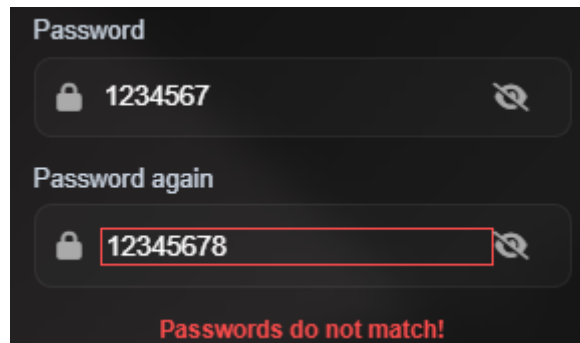
- Bemenet: érvényes név, e-mail cím, telefonszám, két azonos jelszó
- Elvárt eredmény: a fiók létrejön, megerősítő e-mail kiküldésre kerül, az adatok bekerülnek a users gyűjteménybe
- Eredmény: **Megfelelt**



Visszaigazoló email

2. Teszteset: eltérő jelszavak

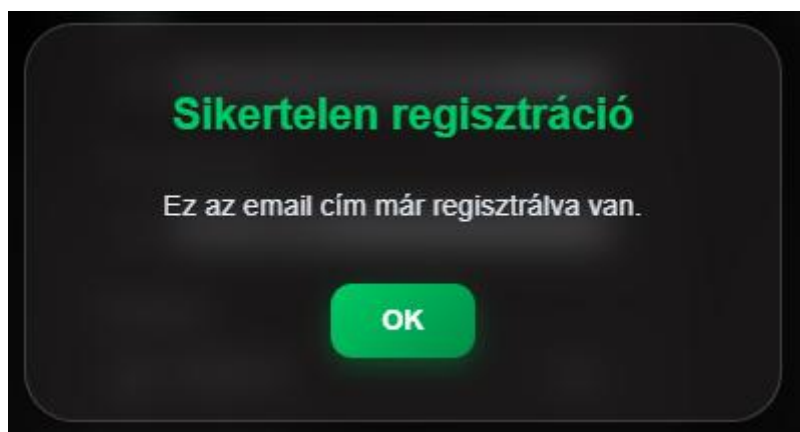
- Bemenet: különböző jelszó és jelszó megerősítés
- Elvárt eredmény: a rendszer nem engedi a regisztrációt, hibajelzés jelenik meg
- Eredmény: Megfelelt



The screenshot shows a dark-themed registration form. It has two input fields: 'Password' with the value '1234567' and 'Password again' with the value '12345678'. Both fields have a lock icon on the left and a toggle icon on the right. Below the fields, a red error message reads 'Passwords do not match!'.

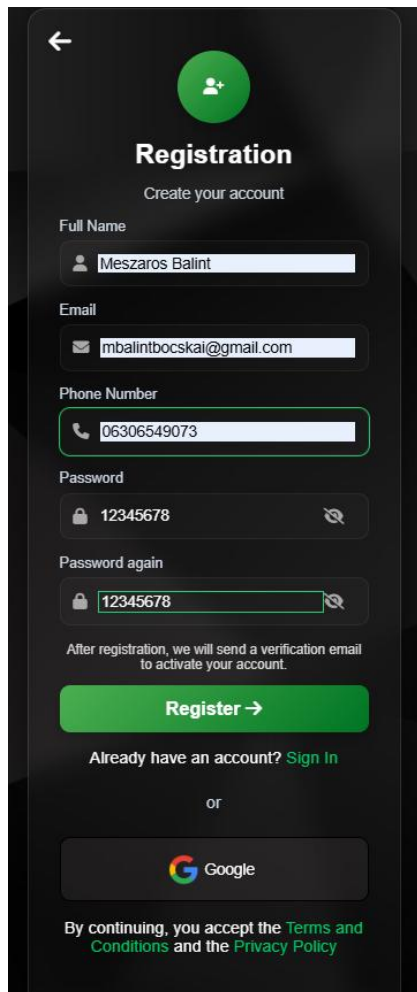
3. Teszteset: már használt e-mail cím

- Bemenet: korábban regisztrált e-mail cím
- Elvárt eredmény: a rendszer tájékoztatja a felhasználót, hogy az e-mail már használatban van
- Eredmény: Megfelelt



4. Teszteset: Google regisztráció

- Bemenet: érvényes Google-fiók
- Elvárt eredmény: sikeres regisztráció és belépés, felhasználói adatok mentése
- Eredmény: **Megfelelt**



```
+ Add field

createdAt: 25 March 2026 at 16:22:27 UTC+1
email: "mbalintbocskai@gmail.com"
emailVerified: true
fullname: "Meszaros Balint"
lastLoginAt: 27 March 2026 at 15:28:55 UTC+1
phone: "Not provided"
provider: "google"
verificationSyncedAt: 27 March 2026 at 18:21:04 UTC+1
```

4. Bejelentkezés

4.1 Funkcionális leírás

A bejelentkezési modul feladata a már regisztrált felhasználók hitelesítése és beléptetése. A rendszer támogatja az e-mail-jelszó alapú bejelentkezést valamint a Google fiókkal való belépést is.

4.1.1 Főbb funkciók:

- E-mail és jelszó alapú bejelentkezés
- Google alapú bejelentkezés
- Jelszó láthatóságának vezérlése
- Elfelejtett jelszó oldal elérése
- Megerősítetlen e-mail cím felismerése
- Megerősítő e-mail újraküldése
- Sikeres belépés után átirányítás a főoldalra
- Hibaüzenetek megjelenítése egyedi modális ablakban

4.2 Frontend felépítés (HTML és CSS)

4.2.1 Felületi elemek (signIn.html)

A bejelentkezési felület egy központi űrlapot tartalmaz, amelyben e-mail cím és jelszó adható meg. Az oldalon külön gomb található a megerősítő e-mail újraküldésére, valamint Google alapú hitelesítésre is. Az űrlap alatt elérhető a regisztrációs oldal és a jelszó-visszaállítási oldal hivatkozása.

4.2.2 Design és reszponzivitás (signIn.css)

A felület vizuálisan illeszkedik a regisztrációs oldalhoz. Az input mezők fókusz esetén kiemelést kapnak, a gombok és ikonok pedig mobilon és asztali nézetben is jól használhatók. A kialakítás célja az átlátható és gyors bejelentkezési folyamat támogatása.

4.3 Programozási logika (signIn.js)

4.3.1 Jelszó láthatóságának kezelése

A jelszómező melletti szem ikon segítségével a felhasználó válthat a rejtett és látható jelszómegjelenítés között. Ez megkönnyíti az adatbevitel ellenőrzését.

```
// --- PASSWORD VISIBILITY TOGGLE ---
if (togglePasswordButton && passwordInput) {
  togglePasswordButton.addEventListener( type: 'click', listener: () => {
    // Switch between password and text input type
    const isPasswordHidden = passwordInput.getAttribute( qualifiedName: 'type' ) === 'password';
    passwordInput.setAttribute( qualifiedName: 'type', value: isPasswordHidden ? 'text' : 'password' );

    // Update icon state (eye / eye-slash)
    togglePasswordButton.classList.toggle( token: 'fa-eye' );
    togglePasswordButton.classList.toggle( token: 'fa-eye-slash' );
  } );
}
```

4.3.2 E-mail-jelszó alapú hitelesítés

A rendszer a signInWithEmailAndPassword függvény segítségével végzi a hitelesítést. Sikeres azonosítás után a felhasználó adatai frissítésre kerülnek, majd a rendszer ellenőrzi, hogy szükséges-e e-mail megerősítés.

4.3.3 E-mail megerősítés ellenőrzése

Ha a felhasználó jelszavas hitelesítésű fiókkal rendelkezik, de az e-mail cím még nincs megerősítve, a rendszer:

- Új megerősítő e-mailt küld
- Kijelentkezteti a felhasználót
- Tájékoztató modális üzenetet jelenít meg
- Nem engedi a továbblépést a védett funkciókhoz

```
if (requiresEmailVerification(user)) {  
  await signOut(auth);  
  await forgeXModal(  
    title: "Email megerosítése szukseges",  
    message: "Az email címed még nincs megerosítve. Kattints a visszaigazoló email újraküldése gombra, ha új levelet szeretnél kenni."  
  );  
  return;  
}
```

4.3.4 Google alapú belépés

A signInWithPopup segítségével a felhasználó Google-fiókkal is bejelentkezhetsz.

A sikeres belépés után a rendszer frissíti vagy létrehozza a felhasználói dokumentumot a Firestore adatbázisban.

4.3.5 Megerősítő e-mail újraküldése

A felhasználó külön gombbal kérhet új megerősítő e-mailt. Ehhez előbb meg kell adnia az e-mail címét és jelszavát. A rendszer ellenőrzi a hitelesítést, majd szükség esetén újra elküldi a visszaigazoló levelet.

4.4 Hibakezelés

A rendszer külön kezeli például:

- Hibás e-mail vagy jelszó
- Túl sok sikertelen próbálkozás
- Érvénytelen e-mail formátum
- Google popup megszakítása
- Megerősítetlen fiók

4.5 Tesztelési napló

1. Teszteset: sikeres bejelentkezés

Bemenet: helyes e-mail és jelszó

Elvárt eredmény: a felhasználó belép és a főoldalra kerül

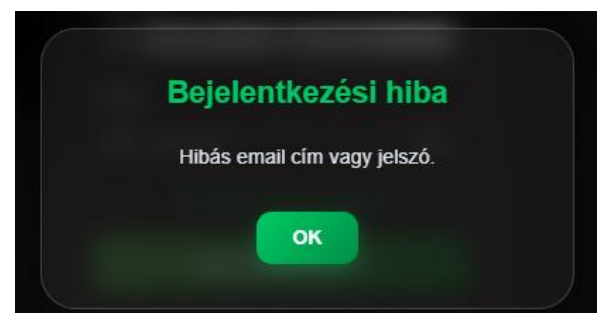
Eredmény: Megfelelt

2. Teszteset: hibás jelszó

Bemenet: helyes e-mail, hibás jelszó

Elvárt eredmény: hibaüzenet jelenik meg, belépés nem történik

Eredmény: Megfelelt



3. Teszteset: nem megerősített e-mail

Bemenet: regisztrált, de nem aktivált fiók

Elvárt eredmény: a rendszer új megerősítő levelet küld és nem engedi a belépést

Eredmény: Nem **Megfelelt**

4. Teszteset: megerősítő e-mail újraküldése

Bemenet: helyes e-mail és jelszó, még nem megerősített fiók

Elvárt eredmény: új visszaigazoló levél kiküldése a felhasználónak

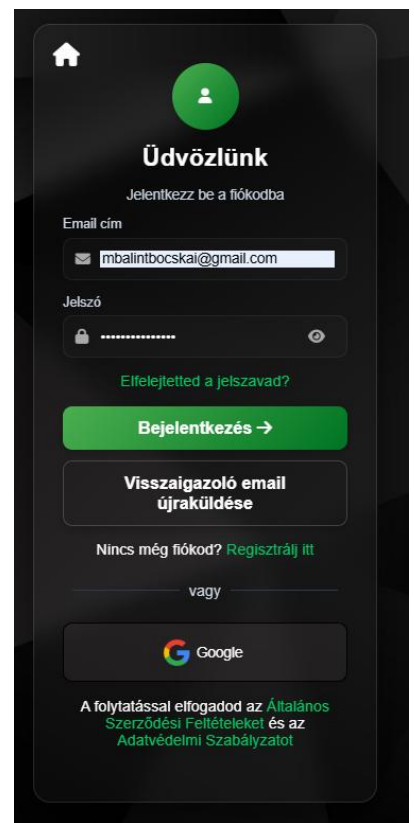
Eredmény: Megfelelt

5. Teszteset: Google belépés

Bemenet: érvényes Google-fiók

Elvárt eredmény: sikeres belépés és felhasználói adatok szinkronizálása

Eredmény: Megfelelt



4.6 Programozási logika (signIn.js)

4.6.1 Jelszó-visszaállítás

4.6.2 Funkcionális leírás

A jelszó-visszaállítási modul célja, hogy a felhasználó elfelejtett jelszó esetén e-mail alapú visszaállító folyamatot kezdeményezhessen.

4.6.3 Főbb funkciók:

- E-mail cím bekérése
- Jelszó-visszaállító e-mail küldése Firebase-en keresztül
- Sikeres és sikertelen folyamatok visszajelzése modális ablakkal
- Küldés alatti gombletiltás

Reset your password

for mbalintbocskai@gmail.com

New password



SAVE

4.7 Frontend felépítés (resetPassword.html)

Az oldal egyszerű felépítésű, egyetlen e-mail mezőt és egy küldő gombot tartalmaz. A cél itt a gyors és egyértelmű használhatóság volt.

4.8 Programozási logika (resetPassword.js)

Az űrlap elküldésekor a rendszer meghívja a `sendPasswordResetEmail` függvényt. Siker esetén tájékoztató modális ablak jelenik meg, hiba esetén pedig hibajelzés látható. A folyamat



ForgeX <ForgeX@forgex-2026.firebaseio.com>

to me ▾

Wed 25 Mar, 15:53 (2 days ago)



Hello,

Follow this link to verify your email address.

https://forgex-2026.firebaseio.com/_/auth/action?mode=verifyEmail&oobCode=WSHds2v9OMIdg6IE-P5jyITDdoccr0luMOle_h9eucAAAGdJXwc8g&apiKey=AlzaSyA-qcUAb6bqjwxftrKBHJOCce0Jw4uGHME&continueUrl=http%3A%2F%2Flocalhost%3A63342%2Fsignin.html%3Fverified%3D1&lang=en

...

↩ Reply

➡ Forward



idejére a gomb letiltásra kerül, hogy ne történjen többszöri küldés.

4.9 Tesztelési napló

1. Teszteset: érvényes e-mail cím

Bemenet: regisztrált e-mail cím

Elvárt eredmény: a rendszer visszaállító levelet küld

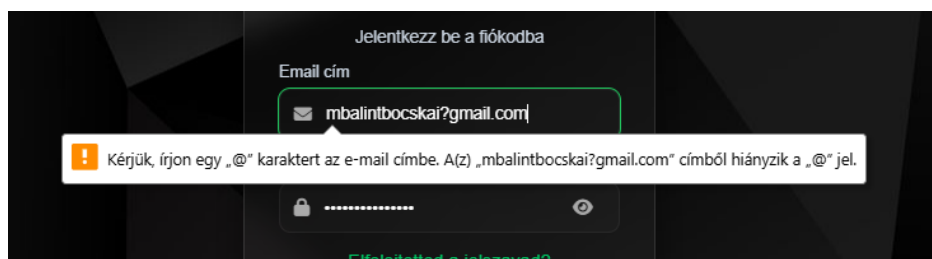
Eredmény: **Megfelelt**

2. Teszteset: hibás e-mail cím

Bemenet: érvénytelen vagy hibás formátumú e-mail

Elvárt eredmény: a rendszer hibaüzenetet jelenít meg

Eredmény: **Megfelelt**



A kliensoldali script moduláris felépítésű, aszinkron hívásokkal kezeli a hálózati kommunikációt.

5. Profil

5.1 Funkcionális leírás

A profil oldal célja, hogy a bejelentkezett felhasználók megtekinthessék a saját adataikat, valamint kezelhessék az aktív foglalásaikat. A modul csak hitelesített felhasználó számára elérhető, és figyelembe veszi az e-mail megerősítés állapotát is.

5.1.1 Főbb funkciók:

- Hozzáférés-védelem bejelentkezett felhasználók számára
- E-mail megerősítés ellenőrzése
- Személyes adatok megjelenítése a Firestore adatbázisból
- Foglalások listázása
- Foglalások törlése
- Biztonságos kijelentkezés
- Helyi kosár- és kedvencállapot törlése kijelentkezéskor

5.2 Frontend felépítés (HTML és CSS)

5.2.1 Felületi elemek (profil.html)

A profiloldal egy központi kártyát jelenít meg, amelyben külön blokkokban láthatók a felhasználó adatai és a foglalások. A `#prof-name`, `#prof-email` és `#prof-phone` elemek dinamikusan töltődnek be, a `#user-bookings` tároló pedig a foglalási lista megjelenítésére szolgál.

5.2.2 Design és megjelenés (profil.css)

A felület letisztult, a projekt többi oldalával egységes vizuális stílust követ. A foglalási elemek külön blokkban jelennek meg, a törlés gomb pedig egyértelműen elkülönül a többi elemtől, jelezve annak funkcióját.

5.3 Programozási logika (profil.js)

5.3.1 Hozzáférés-ellenőrzés

Az `onAuthStateChanged` figyel a felhasználó bejelentkezési állapotát. Ha nincs hitelesített felhasználó, a rendszer automatikusan a bejelentkezési oldalra irányít. Bejelentkezett állapotban a modul további ellenőrzést végez az e-mail megerősítés állapotára is.

5.3.2 Profiladatok betöltése

A rendszer a Firestore users gyűjteményéből tölti be a felhasználóhoz tartozó adatokat, majd megjeleníti azokat a profiloldalon. A betöltött mezők:

- Teljes név
- E-mail cím
- Telefonszám

5.3.3 Foglalások kezelése

A bookings gyűjteményből lekérdezés történik a felhasználó egyedi azonosítója alapján. A találatok dinamikusan jelennek meg a felületen. Minden foglaláshoz törlés gomb tartozik, amely megerősítő modális ablak után végzi el az eltávolítást.

5.3.4 Kijelentkezési folyamat

Kijelentkezéskor a rendszer megerősítést kér a felhasználótól. Jóváhagyás után:

- Kijelentkezteti a felhasználót
- Törli a localStorage-ben tárolt kosár- és kedvencadatokat
- Visszairányítja a főoldalra

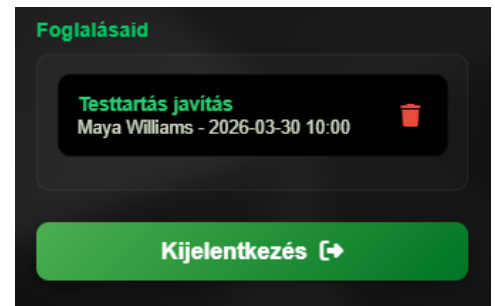
5.4 Tesztelési napló

1. Teszteset: profiloldal megnyitása bejelentkezve

Bemenet: hitelesített felhasználó

Elvárt eredmény: a személyes adatok és foglalások megjelennek

Eredmény: **Megfelelt**

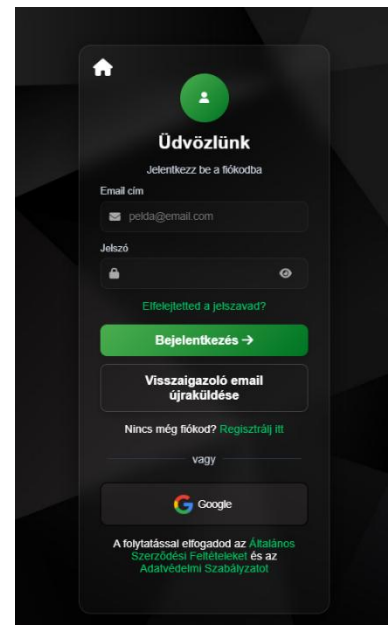


2. Teszteset: profiloldal megnyitása kijelentkezett állapotban

Bemenet: nincs bejelentkezett felhasználó

Elvárt eredmény: átirányítás a bejelentkezési oldalra

Eredmény: **Megfelelt**

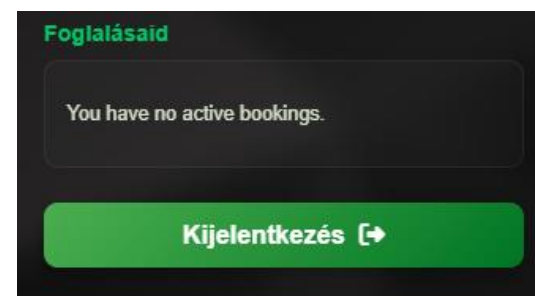


3. Teszteset: foglalás törlése

Bemenet: meglévő foglalás törlése

Elvárt eredmény: a foglalás eltűnik a listából és az adatbázisból

Eredmény: **Megfelelt**



4. Teszteset: üres foglalási lista

Bemenet: olyan felhasználó, akinek nincs foglalása

Elvárt eredmény: tájékoztató üzenet jelenik meg

Eredmény: **Megfelelt**

6. Shop

6.1 Funkcionális leírás

A Shop modul a ForgeX webáruház központi felülete, ahol a felhasználók böngészhetik a termékeket, kereshetnek, kategória szerint, szűrhetnek ár szerint, rendezhetnek, valamint a kiválasztott termékeket kedvencekhez vagy kosárhoz adhatják.

6.1.1 Főbb funkciók:

- Dinamikus termékkártyák megjelenítése
- Keresés név és leírás alapján
- Kategóriaszűrés
- Ár szerinti rendezés
- Részletes termékmodál megnyitása
- Méret, szín vagy kiserelés kiválasztása
- Kedvencekhez adás
- Kosárba helyezés
- Firestore és localStorage szinkronizáció bejelentkezett felhasználó esetén

6.2 Frontend felépítés (HTML és CSS)

6.2.1 Felületi struktúra (shop.html)

Az oldal két fő részre bontható:

- Oldalsó szűrőpanel
- Termékkártyákat tartalmazó rácsos elrendezés

A szűrőpanel tartalmazza a keresőt, a kategóriagombokat és a rendezési lehetőséget. A termékek egy products-grid nevű tárolóban jelennek meg. A részletes terméknézet modális ablakban nyílik meg.

6.2.2 Design és UX (User Experience)((shop.css)

A felület reszponzív kialakítású. Asztali nézetben a szűrőpanel rögzített oldalsávként jelenik meg, kisebb kijelzőn pedig összecsuksukható formában használható. A termékkártyák hover hatást kapnak, a modális ablak pedig külön képi és adatmegjelenítő részből áll.

6.3 Programozási logika (shop.js

6.3.1 Termékadatok kezelése

A termékek adatai egy központi **PRODUCT_DATA** objektumban szerepelnek. Ebben minden termékhez hozzárendelve található:

- Név
- Rövid és hosszú leírás
- Kép vagy képek
- Méret vagy kiszerezés
- Szín vagy íz
- Ár
- Kategória

6.3.2 Keresés, szűrés és rendezés

A rendszer a **getFilteredProductIds()** és **applyFiltersAndSorting()** logikájával kliensoldalon szűri és rendezi a termékeket. A keresés valós időben működik, a kategória kiválasztása és a rendezés pedig azonnal újrendereli a látható elemeket.

6.3.3 Termékmodál működése

A termékkártyára kattintva modális ablak nyílik meg. A modál dinamikusan kezeli:

- Az aktuális termék adatait
- A választható méreteket vagy kiszerezéseket
- A választható színeket vagy ízeket
- Az ár frissítését
- A termékkép módosítását a kiválasztott variáció alapján

6.3.4 Kedvencek és kosár kezelése

A rendszer csak bejelentkezett felhasználó számára engedi a kedvencekhez adást és a kosárba helyezést. Ha a felhasználó nincs belépve, a modul figyelmeztető modális ablakot jelenít meg, majd a bejelentkezési oldalra irányítja.

Bejelentkezett felhasználó esetén:

- A kosár és kedvencek adatai betöltődnek Firestore-ból
- A módosítások localStorage-be is bekerülnek
- A változások visszamentődnek a felhőbe

6.3.5 Reszponzív szűrőlogika

Mobil nézetben a szűrőpanel összecukhatóvá válik. A modul külön figyeli az ablakméretet, és ennek megfelelően változtatja a szűrőpanel állapotát.

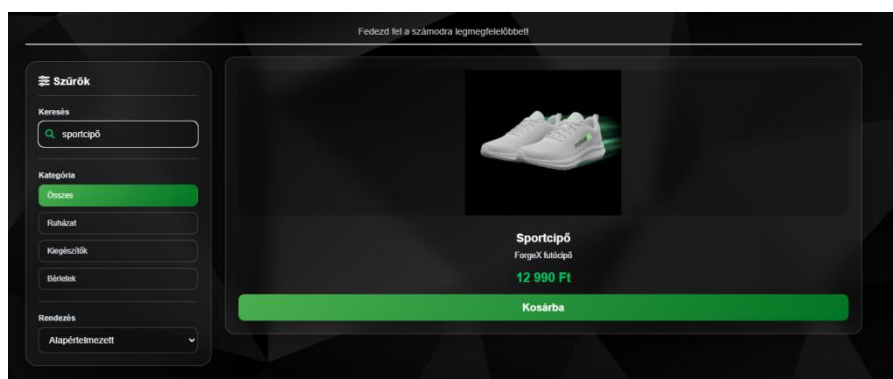
6.4 Tesztelési napló

1. Teszteset: keresés

Bemenet: keresőkifejezés megadása

Elvárt eredmény: csak a megfelelő termékek maradnak láthatók

Eredmény: **Megfelelt**

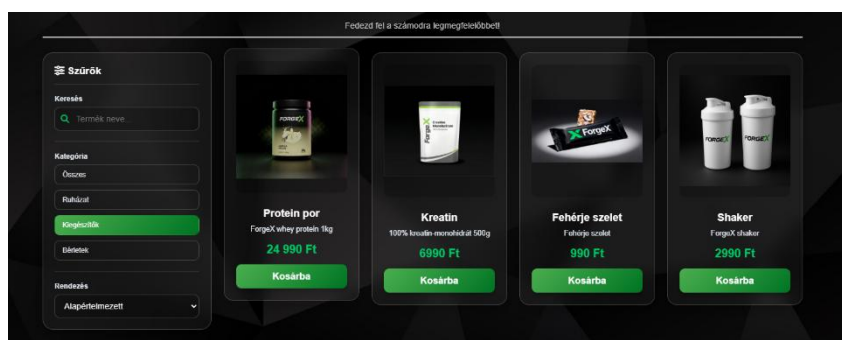


2. Teszteset: kategóriaszűrés

Bemenet: kategóriagomb kiválasztása

Elvárt eredmény: csak az adott kategóriába tartozó termékek jelennek meg

Eredmény: **Megfelelt**

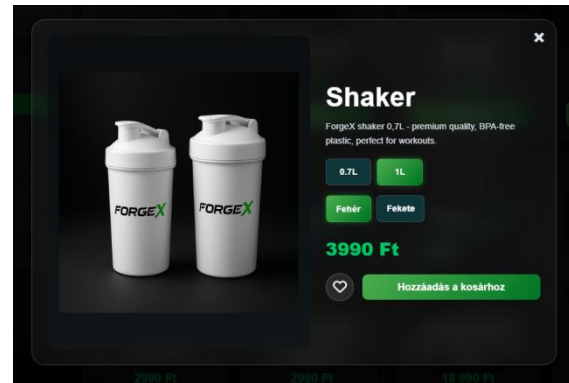


3. Teszteset: termékmodál

Bemenet: termékkártyára kattintás

Elvárt eredmény: megnyílik a modális ablak a termék részleteivel

Eredmény: **Megfelelt**

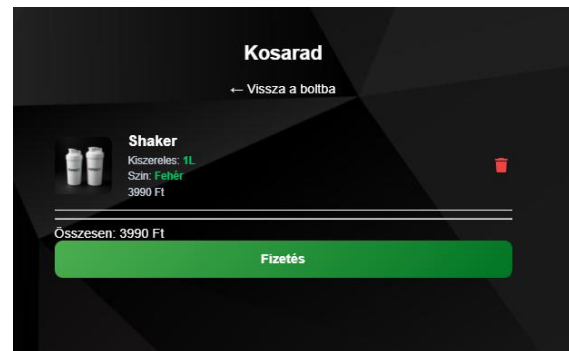


4. Teszteset: kosárba helyezés bejelentkezve

Bemenet: kiválasztott termék kosárba helyezése

Elvárt eredmény: a termék bekerül a kosárba, az adat localStorage-be és Firestore-ba is mentődik

Eredmény: **Megfelelt**

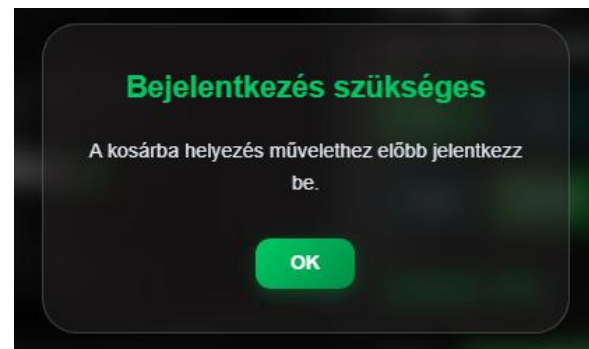


5. Teszteset: kosárba helyezés kijelentkezve

Bemenet: termék kosárba helyezése bejelentkezés nélkül

Elvárt eredmény: figyelmeztető üzenet jelenik meg, majd átirányítás történik

Eredmény: **Megfelelt**



7. Kedvencek

7.1 Funkcionális leírás

A Kedvencek modul célja, hogy a felhasználó elmenthesse a számára érdekes termékeket későbbi megtekintésre vagy gyors kosárba helyezésre. A modul szorosan együttműködik a Shop és a Kosár modulokkal.

7.1.1 Főbb funkciók:

- kedvenc termékek dinamikus listázása
- termékek eltávolítása a kedvencek közül
- Termékek közvetlen kosárba helyezése
- localStorage és Firestore alapú adatkezelés
- üres állapot kezelése

7.2 Frontend felépítés (HTML és CSS)

7.2.1 Felületi szerkezet

A kedvencek oldal a shop oldallal azonos termékkártya-elrendezést használja, így a vizuális megjelenés konzisztens marad. A kártyák a #favorites-list elembe kerülnek dinamikusan, üres lista esetén pedig a #no-favs elem jelenik meg.

7.2.2 Design és megjelenés (favourite.css)

A felületen a kosárba helyezés és az eltávolítás külön akciógombot kapott. A kialakítás célja az volt, hogy a felhasználó gyorsan dönthessen a kedvencek megtartásáról vagy vásárlásra való áthelyezéséről.

7.3 Programozási logika (favourite.js)

7.3.1 Betöltés és megjelenítés

Bejelentkezés után a rendszer lekéri a kedvenceket a Firestore adatbázisból, majd localStorage-ben is eltárolja azokat. A renderFavorites() függvény minden mentett elemhez külön termékkártyát hoz létre.

7.3.2 Kosárba helyezés kedvencekből

A kedvencek oldalról a felhasználó közvetlenül kosárba helyezheti a terméket. Ilyenkor a rendszer:

- Létrehoz egy új kosárelemet
- LocalStorage-be menti
- Firestore-ba is visszazinkronizálja
- Visszajelző modális ablakot jelenít meg

7.3.3 Törlés a kedvencek közül

A rendszer az adott elem egyedi azonosítója alapján törli a kedvencet, frissíti a helyi listát, majd a változást a felhőbe is menti.

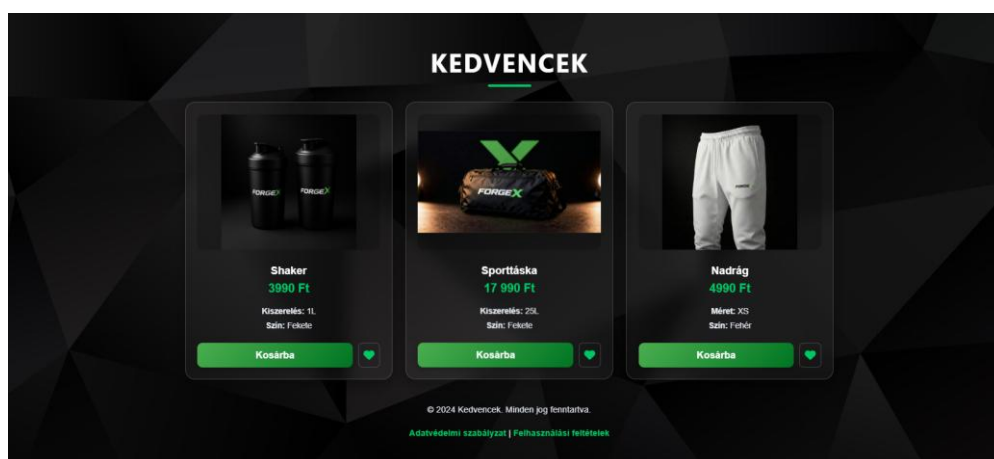
7.4 Tesztelési napló

1. Teszteset: kedvencek betöltése

Bemenet: bejelentkezett felhasználó mentett kedvencekkel

Elvárt eredmény: a kedvenc termékek megjelennek

Eredmény: **Megfelelt**

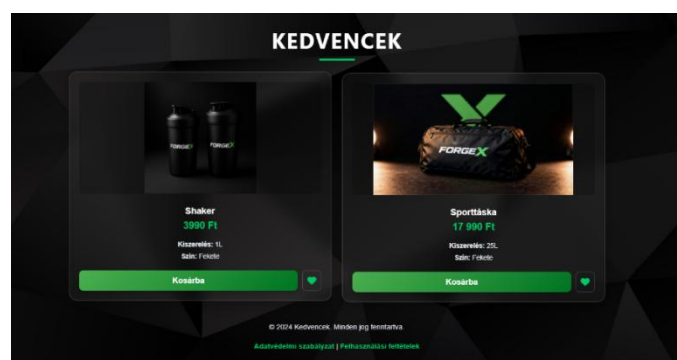


2. Teszteset: eltávolítás kedvencek közül

Bemenet: meglévő kedvenc törlése

Elvárt eredmény: a termék eltűnik a listából és az adatbázisból

Eredmény: **Megfelelt**

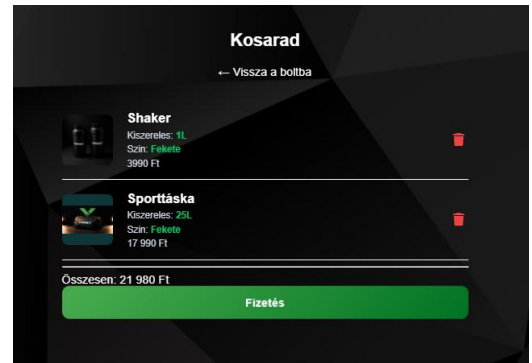


3. Teszteset: kosárba helyezés kedvencekből

Bemenet: kedvenc termék kosárba helyezése

Elvárt eredmény: a termék bekerül a kosárba, a kosárszámláló frissül

Eredmény: **Megfelelt**



4. Teszteset: üres kedvencek lista

Bemenet: nincs mentett kedvenc

Elvárt eredmény: tájékoztató szöveg jelenik meg

Eredmény: **Megfelelt**



8. Kosár

8.1 Funkcionális leírás

A Kosár modul feladata a kiválasztott termékek áttekinthető megjelenítése, az összegzés kiszámítása és a fölösleges tételek eltávolításának biztosítása a fizetés előtt.

8.1.1 Főbb funkciók:

- Kosárban lévő termékek dinamikus megjelenítése
- Termékjellemzők megjelenítése, például méret, szín, kiserelés vagy íz
- Végösszeg automatikus számítása
- Tétel törlése
- Üres kosár állapot kezelése
- Firestore és localStorage szinkronizáció

8.2 Frontend felépítés (HTML és CSS)

8.2.1 Felületi struktúra (cart.html)

Az oldal központi eleme a `#cart-items-container`, amely dinamikusan jeleníti meg a kosár tartalmát. Alatta található az összegző rész a végösszeggel és a fizetés gombbal.

8.2.2 Design és megjelenés (cart.css)

A kialakítás célja a gyors áttekinthetőség. A törlés gomb jól elkülönül, a végösszeg kiemelt helyen jelenik meg, a mobil nézet pedig egyszerűsített, jól olvasható elrendezést használ.

8.3 Programozási logika (cart.js)

8.3.1 Betöltés és szinkronizáció

Bejelentkezés esetén a rendszer a Firestore carts gyűjteményéből betölti az adott felhasználó kosarát, majd localStorage-ben is eltárolja. Kijelentkezett állapotban a helyi kosár törlésre kerül.

8.3.2 Dinamikus renderelés

A `renderCart()` függvény feladata:

- A kosár elemeinek kirajzolása
- A végösszeg kiszámítása
- Az üres állapot kezelése
- A fizetés gomb elrejtése, ha nincs tétel a kosárban

```
renderCart(  
  updatedCart,  
  document.getElementById( elementId: "cart-items-container"),  
  document.getElementById( elementId: "total-price"),  
  document.getElementById( elementId: "checkout-btn")  
);  
};
```

8.3.3 Tétel törlése

A `removeFromCart()` eltávolítja az adott elemet a helyi tárolóból, frissíti a felületet, majd bejelentkezett állapotban az adatbázisban is elvégzi a módosítást.

```
window.removeFromCart = async function (itemId) : Promise<void> {  
  let cartItems = JSON.parse(localStorage.getItem( key: "cart")) || [];  
  cartItems = cartItems.map((item : T ) : any & {...} => ({  
    ...item,  
    image: normalizeImagePath(item.image)  
  }));  
};
```

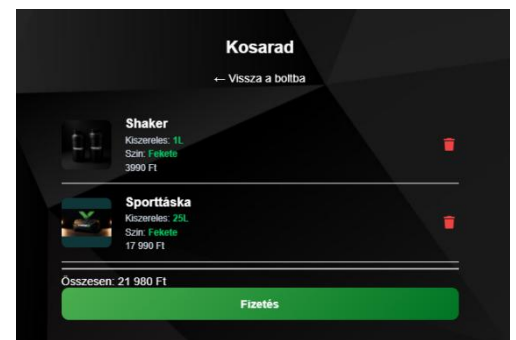
8.4 Tesztelési napló

1. Teszteset: kosár betöltése

Bemenet: bejelentkezett felhasználó meglévő kosárral

Elvárt eredmény: a tételek és a végösszeg megjelenik

Eredmény: **Megfelelt**

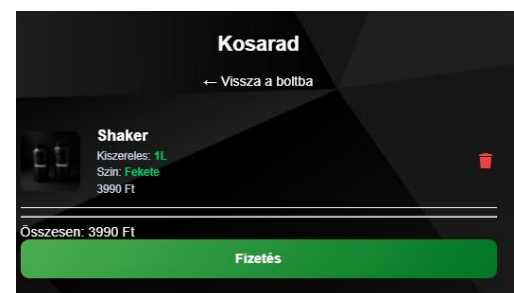


2. Teszteset: tétel törlése

Bemenet: kosárelem törlése

Elvárt eredmény: a tétel eltűnik, az összeg újrászámolódik

Eredmény: **Megfelelt**

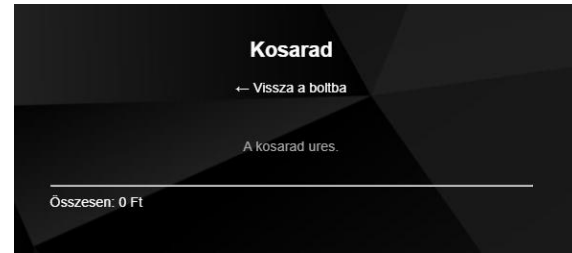


3. Teszteset: üres kosár

Bemenet: nincs kosárban termék

Elvárt eredmény: üres állapot szöveg jelenik meg, a fizetés gomb rejtve marad

Eredmény: **Megfelelt**

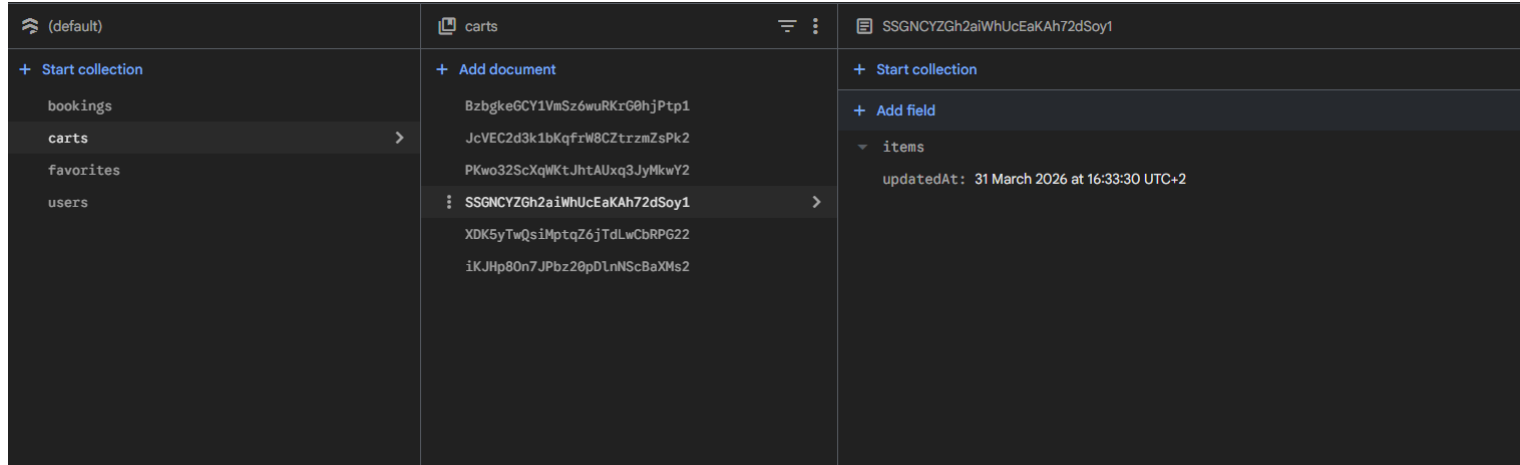


4. Teszteset: adatbázis szinkron

Bemenet: kosár módosítása bejelentkezett állapotban

Elvárt eredmény: a módosítás a Firestore-ban is megjelenik

Eredmény: **Megfelelt**



9. Fizetés

9.1 Funkcionális leírás

A Fizetés modul célja a vásárlási folyamat lezárása. A felhasználó ezen az oldalon ellenőrzi a kosár tartalmát, megadja a szállítási és fizetési adatokat, majd sikeres ellenőrzés után leadja a rendelést.

9.1.1 Főbb funkciók:

- A kosár tartalmának megjelenítése
- Végösszeg kiszámítása és kiírása
- Szállítási adatok bekérése
- Fizetési adatok formai ellenőrzése
- Valós idejű inputkorlátozás számmezőknél
- Sikeres fizetés után a kosár kiürítése
- Visszajelző sikerüzenet megjelenítése

9.2 Frontend felépítés (HTML és CSS)

9.2.1 Felületi felosztás (pay.html)

A fizetési oldal három logikai egységből áll:

- Rendelési összesítő
- Szállítási adatok
- Fizetési adatok

Az oldal tetején a kosár tartalma és a végösszeg jelenik meg, ezután következnek az adatbekérő mezők.

9.2.2 Design és UX (pay.css)

A megjelenés illeszkedik a projekt többi oldalához. A hibás mezők piros keretet kapnak, a hozzájuk tartozó hibaüzenet pedig közvetlenül a mező alatt jelenik meg. Mobil nézetben a többoszlopos mezők egymás alá rendeződnek.

9.3 Programozási logika (pay.js)

9.3.1 Rendelési adatok betöltése

A modul a localStorage-ben tárolt kosárelemekből építi fel a rendelési összesítőt. Az egyes elemeknél külön megjelennek a termékjellemzők is, például méret, szín vagy kiserelés.

9.3.2 Mezők validálása

A rendszer ellenőrzi:

- Teljes név
- E-mail cím formátuma
- Város
- Irányítószám
- Cím
- Bankkártyaszám
- Lejárat dátum
- CVV

A validáció részben reguláris kifejezésekkel történik, részben pedig valós idejű karakterkorlátozással.

9.3.3 Sikeres fizetés utáni folyamat

Ha minden adat megfelel az elvárásoknak, a rendszer:

- Kiüríti a localStorage kosarat
- Nullázza a kosárjelzőt
- Bejelentkezett felhasználónál a Firestore carts dokumentumot is üríti
- Elrejt az űrlapot
- Megjeleníti a sikeres rendelés visszaigazolását

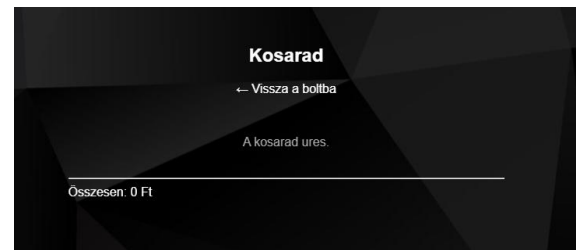
9.4 Tesztelési napló

1. Teszteset: helyes adatokkal történő fizetés

Bemenet: minden mező helyesen kitöltve

Elvárt eredmény: sikeres visszaigazolás, a kosár kiürül

Eredmény: **Megfelelt**



2. Teszteset: hibás e-mail cím

Bemenet: hibás e-mail formátum

Elvárt eredmény: a rendszer hibajelzést ad

Eredmény: **Megfelelt**

3. Teszteset: hibás bankkártyaszám

Bemenet: 16 számjegynél rövidebb vagy nem numerikus adat

Elvárt eredmény: a rendszer nem engedi a fizetést

Eredmény: **Megfelelt**

4. Teszteset: sikeres fizetés után kosárürítés

Bemenet: sikeres rendelés

Elvárt eredmény: a kosár localStorage-ben és Firestore-ban is kiürül

Eredmény: **Megfelelt**

FIZETÉSI ADATOK

123451234512345

16 számjegyű kártyaszám szükséges!

10. Edzéstervek és Gyakorlatok

10.1 Funkcionális leírás

Az Edzéstervek modul egy korosztály-specifikus fitness megoldásokat kínál. A felhasználók interaktív kártyákon keresztül választhatják ki a számukra ideális programot, amelyet AI által generált demonstrációs videók tesznek érthetővé.

10.1.1 Főbb funkciók:

- **Korosztály alapú kategorizálás:** 10 évestől a 60+ korosztályig terjedő, szakmailag differenciált edzéstervek (Junior, Felnőtt, Master, Senior).
- **AI-vezérelt Videó Demonstráció:** Minden tervhez egyedi, mesterséges intelligenciával generált videó tartozik, amely bemutatja a gyakorlatok helyes végrehajtását.
- **Dinamikus Modal Rendszer:** A részletek (videó, ár, pontos célok) egy felugró ablakban (Pop-up) jelennek meg, anélkül, hogy a felhasználónak el kellene hagynia az oldalt.
- **E-commerce integráció:** Az edzéstervek digitális termékként közvetlenül a kosárba helyezhetők, szinkronizálva a helyi tárolóval (LocalStorage) és a felhőalapú adatbázissal (Firestore).

10.2 Frontend felépítés (HTML és CSS)

10.2.1 Szerkezeti felépítés (workouts.html)

Az oldal struktúrája a letisztult kártya-alapú dizájnnra épül:

1. **Szekció Fejléc (.section-header):** Tartalmazza a főcímet és a márkára jellemző neonzöld elválasztó vonalat (.divider-line).
2. **Kártya Konténer (.age-cards-container):** Flexbox alapú elrendezés, amely összefogja az .age-card elemeket.
3. **Adatvezérelt Kártyák:** Minden kártya rendelkezik egy data-modal attribútummal, amely összeköti a HTML elemet a JavaScript adatbázisával.
4. **Információs Modal (#infoModal):** Egy rejtett réteg, amely a videólejátszót (video) és a dinamikus vásárlási gombot (.add-to-cart-btn) tartalmazza.

10.2.2 Megjelenés és technikai optimalizáció (workouts.css)

A CSS megoldások fókuszában a vizuális stabilitás és a modern megjelenés áll:

Hardveres gyorsítás: A kártyák transform: translate3d(0,0,0) és backface-visibility: hidden tulajdonságokat kaptak, ami kényszeríti a GPU használatát.

Glassmorphism 2.0: A mobil nézetben a kártyák háttere sötétebb (rgba(20,20,20,0.85)), hogy a blur effekt ne rontsa az olvashatóságot és a teljesítményt.

CTA Animáció: A "Kosárba teszem" csillogó effektet (::after pseudo-elem) és ruganyos (cubic-bezier) mozgást kapott hover állapotban.

10.3 Programozási logika (workouts.js)

10.3.1 Modális vezérlés és Adatkezelés

A script egy központi `infoData` objektumból dolgozik, amely tartalmazza a videók elérési útját, az árakat és az egyedi azonosítókat.

- **Megnyitás:** A `openModal(type)` függvény dinamikusan cseréli a videó forrását (`source.src`) és újratölti a lejátszót a `video.load()` módszerrel.
- **Bezárás:** A `closeModal()` esemény megállítja a lejátszást és alaphelyzetbe állítja az időkódot, elkerülve a háttérben futó média erőforrás-pazarlását.

10.3.2 Kosár szinkronizáció

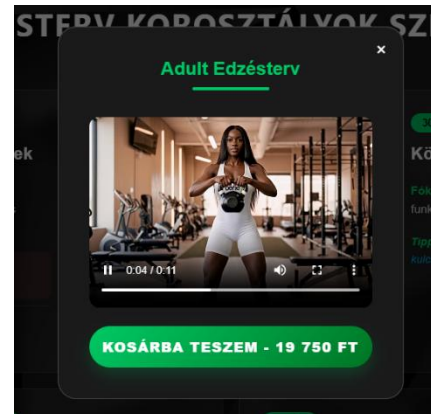
A digitális termékek kezelése egy hibrid folyamat:

1. **Bejelentkezés ellenőrzése:** A rendszer a Firebase Auth segítségével validálja a felhasználót.
2. **Firestore szinkron:** Minden kosárba tétel után a script frissíti a `carts/{uid}` dokumentumot a Firebase-ben, így az edzésterv minden eszközön elérhető marad.
3. **UI Frissítés:** A fejlécben található kosár-számláló (`cart-count`) azonnal reagál a változásra.

10.4 Tesztelési napló

1. Teszteset: AI Videó és Modális ablak működése

- **Bemenet:** Kattintás egy korosztály kártyára (pl. 17-30 év).
- **Elvárt eredmény:** Az #infoModal megjelenik, a modalTitle dinamikusan frissül a kártya nevére, a hozzárendelt AI-videó pedig automatikusan betöltődik és elindul.
- **Eredmény:** Megfelelt

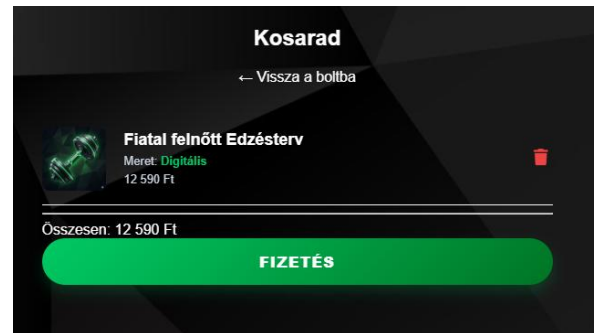


2. Teszteset: Jogosultságkezelés vásárlásnál

- **Bemenet:** Kattintás a "Kosárba teszem" gombra bejelentkezés nélkül.
- **Elvárt eredmény:** A rendszer detektálja a currentUserId hiányát, a forgeXModal figyelmeztető üzenetet küld, majd a felhasználót a signIn.html oldalra navigálja.
- **Eredmény:** Megfelelt

3. Teszteset: Digitális termék adatstruktúra

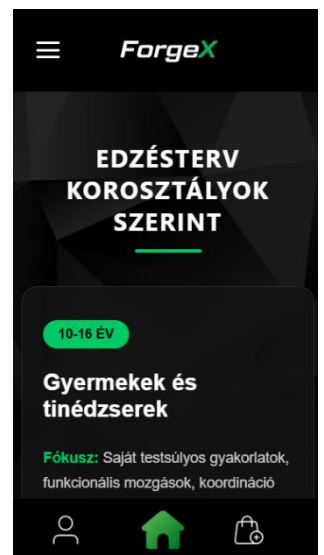
- **Bemenet:** Termék kosárba helyezése bejelentkezett állapotban.
- **Elvárt eredmény:** A localStorage-be kerülő objektum tartalmazza a "Digitális" méretmegjelölést, az egyedi időbélyeg alapú ID-t és a helyes árat, miközben a cart-count azonnal frissül.
- **Eredmény:** **Megfelelt**



4. Teszteset: Mobil nézet stabilitása

Bemenet: Nézet szélességének csökkentése 650px alá.

- **Elvárt eredmény:** A kártyák felveszik a 100%-os szélességet, a belső padding értékek megnőnek a könnyebb érinthetőség érdekében, és a backdrop-filter nem okoz lassulást a görgetésnél.
- **Eredmény:** **Megfelelt**



5. Teszteset: Erőforrás-kezelés bezáráskor

- Bemenet: A modális ablak bezárása az "X" gombbal vagy a háttérre kattintva.
- Elvárt eredmény: A closeModal függvény lefut, a videó lejátszása azonnal megáll (video.pause()), és az idő kód visszaáll nullára, felszabadítva a sáv szélességet.
- Eredmény: Megfelelt

EDZÉSTERV KOROSZTÁLYOK SZERINT

10-16 ÉV

Gyermekek és tinédzserek

Fókusz: Saját testsúlyos gyakorlatok, funkcionális mozgások, koordináció és technika.

Kerülni: A túl nagy súlyokat, a gerinc túlterhelését.

Cél: Alapozás, mozgáskultúra fejlesztése.

17-30 ÉV

Fiatal felnőttek

Fókusz: Nagyobb volumen, intenzitás, súlyzós edzés, erő- és izomnövelés.

Regeneráció: Gyorsabb, heti 4-5 edzés is belefér.

Cél: Maximális erő- és izomépités.

30-45 ÉV

Középkorosztály

Fókusz: Fenntartás, egészségmegőrzés, funkcionális edzés és kardió egyensúly.

Tipp: Ügyelni az ízületekre, a bemelegítés kulcsfontosságú.

45+ ÉV

Negyven felett

Fókusz: Ízületkímélő gyakorlatok, kontrollált mozgások, piramis edzés.

60+ ÉV

Idősebb korosztály

Fókusz: Funkcionális erő, egyensúly, mobilitás.

11. Edzőtermek

11.1 Funkcionális leírás

Az **Edzőtermek modul** célja a ForgeX globális hálózatának (New York, Hayoto, Budapest) bemutatása és az egyes helyszínek egyedi hangulatának, felszereltségének és szolgáltatásainak részletezése. A felület segíti a felhasználót a számára legmegfelelőbb edzőkörnyezet kiválasztásában.

11.1 Főbb funkciók:

- **Helyszínválasztó gyűjtőoldal:** Vizuális kedvcsináló videóval és kártyás elrendezéssel a különböző termekhez.
- **Egyedi terem adatlapok:** Minden helyszín (Budapest, Hayoto, New York) saját aloldallal rendelkezik, amely részletes bio-leírást és specifikációkat tartalmaz.
- **Interaktív képgaléria:** Minden teremnél manuálisan vezérelhető, fade-animációval ellátott képnézegető található.
- **Kapcsolati adatok:** Közvetlen hívás és e-mail küldési lehetőség (tel és mailto linkek), valamint pontos helyszín megjelölés.
- **Szolgáltatáslista:** Ikonokkal illusztrált előnyök és fókuszterületek felsorolása.

11.2 Frontend felépítés (HTML és CSS)

11.2.1 Szerkezeti felépítés (gym.html és aloldalai)

A modul egy gyűjtőoldalból (gym.html) és három specifikus adatlapból áll.

- **Gyűjtőoldal:** Egy welcome-container-rel indul, amely egy sötétített videós hátteret és egy rögzített horgonyra (#gyms) mutató navigációs gombot tartalmaz. A termek kártyái egy Flexbox alapú gym-container-ben kaptak helyet.

- **Adatlapok:** Egy kettéosztott elrendezést (gym-split-layout) használnak. A bal oldalon (gym-image-side) található a vizuális profil (slider + kontaktok), a jobb oldalon (gym-info-side) pedig a szöveges tartalom és a "Glassmorphism" stílusú bio-szekció.

11.2.2 Megjelenés és reszponzivitás (gym.css, style.css)

A design követi a márka futurisztikus, sötét tónusú vonalvezetését.

- **Vizuális effektek:** A teremkártyák hover állapotban megemelkednek (translateY(-10px)), a bennük lévő képek pedig finoman belenagyítódnak (scale(1.1)).
- **Üveghatás:** Az információk egy backdrop-filter: blur(10px) és félátlátszó fehér háttér segítségével különülnek el a sötét háttértől.
- **Mobil optimalizálás:** 900px alatti szélességnél a kettéosztott elrendezés egymás alá rendeződik (flex-direction: column), a kártyák és a szöveges blokkok szélessége pedig kitölti a rendelkezésre álló teret a könnyebb olvashatóság érdekében.

11.3 Kliensoldali logika (JavaScript)

11.3.1. Felhasználói interakciók kezelése

Az oldalon található navigációs elemek (például a "Részletek" gomb) a scroll-behavior: smooth CSS tulajdonság segítségével biztosítanak zökkenőmentes görgetést a horgonypontokhoz. A fejléc és lábléc globális logikáját a modulárisan behúzott auth-header.js és script.js kezeli.

11.3.2 Galéria vezérlés (gym.js)

A termék aloldalain található slider működéséért a `changeSlide` függvény felel:

1. Lekéri az összes `.slide` osztályú elemet.
2. Az aktuális képről eltávolítja az `active` osztályt (megszüntette az opacitást).
3. Kiszámítja az új indexet a modulo (%) operátor segítségével, ami lehetővé teszi, hogy az utolsó kép után az első, az első előtt pedig az utolsó kép jelenjen meg (végtelenített körforgás).
4. Az új elemre ráhelyezi az `active` osztályt, elindítva a CSS-ben definiált 0.5 másodperces fade átmenetet.

11.4 Kontakt és Külső Integrációk

A kontakt szekcióban használt ikonok a FontAwesome könyvtárból származnak. A Google Translate integráció (`googleTranslateElementInit`) lehetővé teszi a specifikus bio-leírások azonnali fordítását a külföldi látogatók számára is.

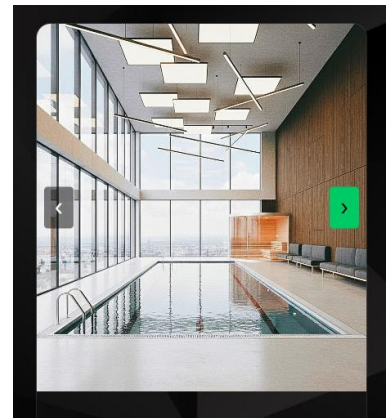
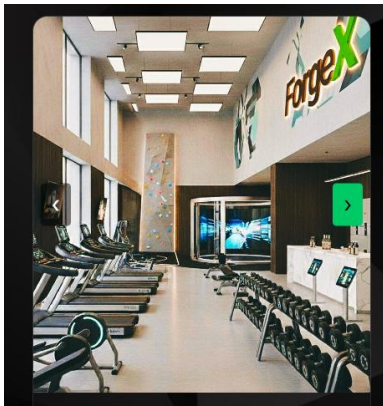
4. Tesztelési napló

1. Teszteset: Horgony-navigáció ellenőrzése

- **Bemenet:** Kattintás a "Részletek ▼" gombra a kezdő videónál.
- **Elvárt eredmény:** Az oldal finoman legördül a termék felsorolásához.
- **Eredmény:** **Megfelelt**

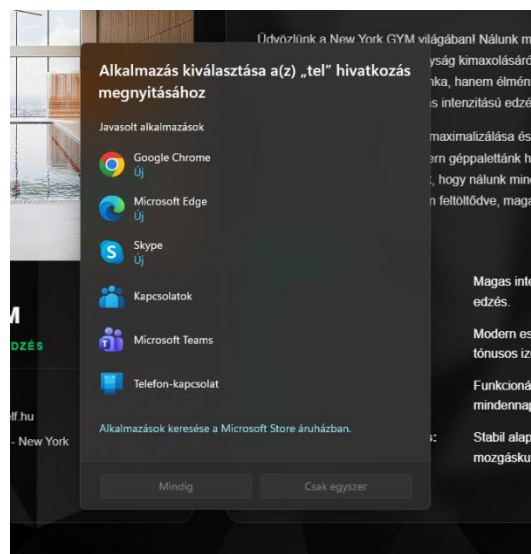
2. Teszteset: Galéria léptetés

- **Bemenet:** Kattintás a "Next" (➤) gombra a Budapest GYM adatlapon.
- **Elvárt eredmény:** Az aktuális kép elhalványul, és megjelenik a következő kép a sorban.
- **Eredmény:** **Megfelelt**



3. Teszteset: Kontakt hivatkozások

- **Bemenet:** Kattintás a telefonszámra mobilon.
- **Elvárt eredmény:** A készülék felajánlja a hívásindítást a megadott számra.
- **Eredmény:** **Megfelelt**



12. Edzői Rendszer (Trainers & Profiles)

12.1 Funkcionális leírás

Az edzői modul célja, hogy bemutassa a ForgeX "SuperSelf" csapatát, és közvetlen kapcsolatot teremtsen a szakemberek és a felhasználók között. A rendszer egy átlátható grid elrendezésből és négy, szerkezetileg azonos, de tartalmában egyedi profiloldalból áll.

12.1.1 Főbb funkciók:

- **Szakmai Portfólió:** Négy különböző specializációval rendelkező edző (Maya Williams, Heath, Hannah Montana és Sensei Hayoto) bemutatása.
- **Egységes Design Architektúra:** Bár minden edző saját aloldallal rendelkezik (maya.html, heat.html, hannah.html, hayoto.html), a forráskód felépítése, a CSS osztályok és a vizuális elrendezés mind a négy esetben megegyezik, biztosítva a brand integritását.
- **Személyre szabott Profiloldalak:** Az edzők egyéni biográfiája, mottója és specializációi a közös stíluslap alapján jelennek meg (példaként bemutatva Sensei Hayoto oldalán keresztül).
- **Intelligens Időpontfoglalás:** A profiloldalokról indított foglalás paraméterezve érkezik a foglalási rendszerbe (booking.html?edzo=nev), így a kiválasztott szakember neve automatikusan átadódik.

12.2 Frontend felépítés (HTML és CSS)

12.2.1 Gyűjtőoldal és Profil szerkezet

A modul két fő strukturális egységre oszlik:

1. **Edzők Grid (trainers.html):** A .trainers-grid osztály egy modern, auto-fit alapú rácsrendszert használ, amely automatikusan rendezi az edzői kártyákat.
2. **Split-Layout Profilok (Példa: hayoto.html):** Az edzői aloldalak aszimmetrikus elrendezést kaptak, amely a többi edzőnél is pontosan így épül fel:
 - **Bal oldal (trainer-image-side):** Tartalmazza a .trainer-profile-box-ot, benne a profilképpel és a közvetlen elérhetőségekkel.
 - **Jobb oldal (trainer-info-side):** A .glass-content blokkban található az edző hitvallása (mottó), a biográfia és a szakmai lista.

12.2.2 Vizuális stílusjegyek (trainers.css)

- **Kép-animációk:** A kártyákon az egér ráhúzásakor a kép finoman nagyítódik (scale(1.1)), a kártya pedig megemelkedik.
- **Glassmorphism profil:** Az információs blokk backdrop-filter: blur(10px) hatást használ, ami minden edzői profilnál egységes mélységet ad a tartalomnak.
- **Tipográfiai hierarchia:** Az ikonikus .gold-quote idézőjel és a hangsúlyos mottó minden profilnál a vizuális fókuszpontot alkotja.

12.3 Programozási logika és Navigáció

12.3.1 Dinamikus útvonalválasztás

A navigáció statikus linkekre épül, de a felhasználói élményt script támogatja:

- **URL Paraméterezés:** Minden edző foglалás gombja az adott edző nevére hivatkozik (pl. ?edzo=hayoto, ?edzo=maya). Ezt a foglалási oldal JS kódja dolgozza fel.
- **Kapcsolatfelvételi automatizmus:** Az e-mail hivatkozások a Google Mail felületére mutatnak, az edzők egyéni címeivel előre kitöltve.

12.4 Tesztelési napló (Hayoto profilja alapján modellezve)

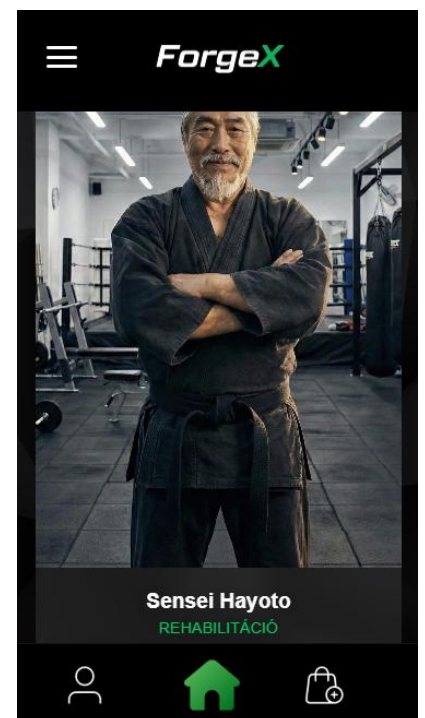
1. Teszteset: Kártya interakció és navigáció

- Bemenet: Egérmutató mozgatása egy edző kártyája fölé, majd kattintás.
- Elvárt eredmény: A kártya megemelkedik, a kép nagyítódik, kattintás után pedig az adott edzőhöz tartozó .html fájl töltődik be.
- Eredmény: **Megfelelt**



2. Teszteset: Profiloldal reszponzivitás

- Bemenet: Nézet szélesség csökkentése 900px alá bármelyik edző profiljánál.
- Elvárt eredmény: A split-layout egymás alá rendeződik, a profilkép felülre kerül, az információk pedig alá.
- Eredmény: **Megfelelt**



3. Teszteset: Időpontfoglalási hivatkozás

- Bemenet: Kattintás az "Időpont Foglalás" gombra Sensei Hayoto (vagy bármelyik másik edző) oldalán.
- Elvárt eredmény: A böngésző a `booking.html?edzo=[nev]` címre navigál, helyesen átadva az azonosítót.
- Eredmény: **Megfelelt**



4. Teszteset: Elérhetőségek működése

- Bemenet: Kattintás az egyéni e-mail hivatkozásokra.
- Elvárt eredmény: Megnyílik a levelezőrendszer a megfelelő edző címével (@superself.hu).
- Eredmény: **Megfelelt**

New Message

hayoto.sensei@superself.hu

Subject

13. Időpontfoglalás

13.1 Funkcionális leírás

Az időpontfoglaló modul célja, hogy a ForgeX rendszerében regisztrált felhasználók számára gyors és átlátható módon biztosítsa a személyi edzések lefoglalását. A folyamat egy háromlépcsős interaktív felületen (stepper) vezeti végig a felhasználót.

13.1.1 Főbb funkciók:

- **Edzőválasztás:** Az elérhető szakemberek profiljainak (kép, név, elérhetőség, helyszín) böngészése és kiválasztása.
- **Dinamikus órakínálat:** A kiválasztott edző szakterületének megfelelő edzéstípusok automatikus betöltése.
- **Interaktív naptárkezelés:** Egyedi fejlesztésű naptárfelület a napok kiválasztásához, a múltbéli dátumok automatikus letiltásával.
- **Idősáv-kezelés:** Fix időközönkénti időpontok (slotok) megjelenítése és foglalása.
- **Profiladatok automatikus szinkronizálása:** A bejelentkezett felhasználó nevének és e-mail címének előtöltése a Firestore adatbázisból.
- **Foglalási adatok rögzítése:** A kiválasztott edző, időpont és kiegészítő adatok (testsúly, életkor, megjegyzés) mentése a Firebase adatbázisba.
- **Hitelesítési kontroll:** Csak megerősített e-mail címmel rendelkező felhasználók számára engedélyezett a véglegesítés.

13.2 Frontend felépítés (HTML és CSS)

13.2.1 Szerkezeti felépítés (booking.html)

A regisztrációs folyamathoz hasonlóan itt is egy központi konténer (**booking-container**) fogja össze az oldalt, de a struktúra három, egymástól jól elkülöníthető részre tagolódik.

Edzőválasztó szekció (trainer-select-grid**):** A felhasználó ezen a szekción belül tudja kiválasztani a számára szimpatikus edzőt. Minden kártya tartalmazza az edző profilképét, nevét és egy részletes elérhetőségi listát ikonokkal ellátva. A szerkezet alapja a **label** és a rejtett **radio** input párosa, ami lehetővé teszi, hogy a teljes kártya felülete kattintható legyen.

Időpont- és óraválasztó szekció (datetime-picker-wrapper**):** Miután kiválasztottuk a nekünk tetsző edzésprogramot, a felület mérete megnő, és két oszlopra oszlik fel: baloldalon helyezkedik el a dinamikusan generált naptár (**calendar-side**), a jobb oldalon pedig az elérhető időszavok listája (**slots-side**).

Személyes adatok űrlap (booking-form**):** A teljes név és email cím után egy rácsos elrendezés (**form-row-grid**) következik az életkor és testsúly számára, végül egy nagyobb szöveges terület (**textarea**) a megjegyzéseknek.

13.2.2 Megjelenés és reszponzivitás (booking.css)

- **Interaktív Edzőkártyák:** A `.trainer-info` osztály kapott egy finom keretet és háttérelmosást. A CSS szelektorok segítségével (`:hover`) a kártyák kiemelkednek, ha az egér föléjük ér, kiválasztáskor pedig a neon zöld keret jelzi a sikeres interakciót. Az edzők képei kör alakú maszkolást kaptak az egységes megjelenés érdekében.
- **Naptár Design:** A naptár nem táblázatos, hanem modern Grid elrendezést használ. A napok (`calendar-day`) négyzetes alakúak, a mai napot egy vékony zöld keret (`.today`), a kiválasztott napot pedig teli zöld háttér jelöli. A múltbéli napok halványított megjelenése vizuálisan is jelzi, hogy azok nem választhatóak.
- **Reszponzív transzformáció: * Asztali nézetben:** Az elemek kényelmesen, egymás mellett vagy rácsban helyezkednek el, kihasználva a rendelkezésre álló szélességet.
 - **Mobil nézetben (@media 768px):** A rendszer automatikusan "stack" (egymás alatti) elrendezésre vált. Az edzőkártyák függőlegesen lesznek, a naptár és az idősávok pedig egymás alá kerülnek.

13.3 Programozási logika (booking.js)

13.3.1 Dinamikus tartalomkezelés

```
const trainerSpecs = {  
  'hayato': ["Post-traumás rehabilitáció", "Ízületi mobilitás & gerinc", "Mindfulness & légzés", "Korrektív gyakorlatok"],  
  'maya': ["Testtartás javítás", "Súlykontroll és alakformálás", "Funkcionális köredzések", "Kezdők felépítése az alapoktól"],  
  'hannah': ["HIIT (Intervallum edzés)", "Szálkásítás és alakformálás", "Állóképesség fejlesztés", "Dinamikus nyújtás és core"],  
  'heath': ["Erőemelő felkészítés", "Izomtömeg-növelés", "Táplálkozási tanácsadás", "Periodizált edzéstervezés"]  
};
```

A rendszer a `trainerSpecs` objektumban tárolja az edzőkhöz tartozó specifikus órákat.

Amikor a felhasználó kiválaszt egy edzőt, a `renderCourses` függvény azonnal frissíti a második lépés kínálatát. Az időpontok kiválasztása (`#time-selection-area`) csak akkor válik láthatóvá, ha az órátípus már ki van jelölve, ezzel biztosítva a logikai sorrendet.

```
function renderCourses(trainerKey) { Show usages  Haas Dániel +1
  courseContainer.innerHTML = '';
  timeArea.style.display = 'none';
  trainerSpecs[trainerKey].forEach((spec, index) => {
    const label = document.createElement( tagName: 'label' );
    label.className = 'course-option';
    label.innerHTML = `
      <input type="radio" name="course" value="course-${index}">
      <div class="course-box">${spec}</div>
    `;
    label.querySelector( selectors: 'input' ).addEventListener( type: 'change', listener: function() {
      timeArea.style.display = 'block';
      renderCalendar();
      renderSlots();
    });
    courseContainer.appendChild(label);
  });
}
```

13.3.2 Egyedi naptárlogika

A naptár generálását a `renderCalendar` függvény végzi, amely az aktuális rendszerdátumot veszi alapul. A logika kiszámolja az adott hónap napjait, és összeveti azokat a mával: a múltbéli napok a `.past` osztályt kapják, így vizuálisan szürkék és funkcionálisan kattinthatatlanok maradnak. A kiválasztott napot a rendszer állapotváltozóban (`selectedDate`) tárolja a véglegesítésig. (A függvényt a `booking.js`-kódban, a 78-adik sortól található meg).

13.3.3 Felhasználói adatok integrációja

Az `onAuthStateChanged` figyelő segítségével a modul felismeri a belépett felhasználót. A Firebase Auth UID alapján a rendszer lekéri a Firestore `users` gyűjteményéből a felhasználó teljes nevét és e-mail címét, majd ezeket automatikusan beírja az űrlap megfelelő mezőibe. Ez csökkenti a felhasználói terhet és javítja az adatok pontosságát.

```
import { onAuthStateChanged } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js";
```

13.3.4 Foglalási folyamat és mentés

A "Foglalás megerősítése" gomb megnyomásakor a rendszer ellenőrzi az adatok teljességét. A foglalás adatai (edző, választott óra, dátum, időpont, valamint a felhasználó fizikai paraméterei és megjegyzései) egy új dokumentumként kerülnek mentésre a Firestore adatbázisba, kiegészítve egy szerveroldali időbélyeggel (`serverTimestamp`).

```
import { doc, getDoc, collection, addDoc, serverTimestamp }  
from "https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js";
```

13.4 Tesztelési napló

1. Teszteset: Dinamikus óralista frissítése

- **Bemenet:** Edző kiválasztása (pl. "Maya Williams")
- **Elvárt eredmény:** A második lépésben csak az ő szakterületéhez tartozó órák jelennek meg.
- **Eredmény:** **Megfelelt**

Válassz órát és időt

Testtartás javítás

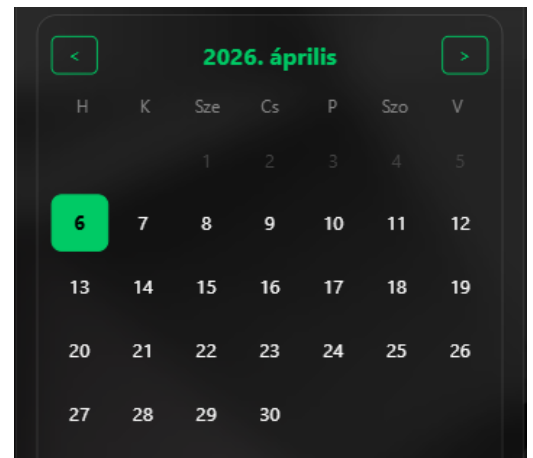
Súlykontroll és alakformálás

Funkcionális köredzések

Kezdők felépítése az alapoktól

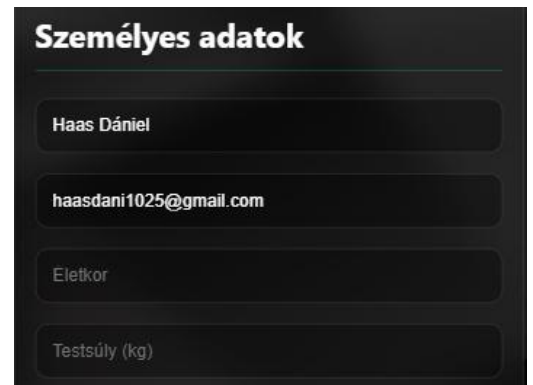
2. Teszteset: Múltbéli dátum letiltása

- **Bemenet:** Kattintás egy tegnapi napra a naptárban.
- **Elvárt eredmény:** A rendszer nem jelöli ki a napot, nem jelennek meg hozzá időszávok.
- **Eredmény:** **Megfelelt**



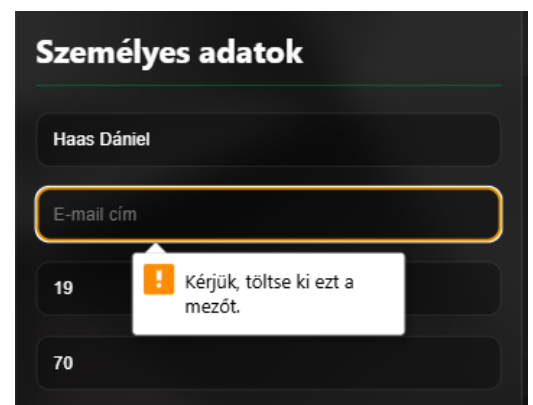
3. Teszteset: Automatikus profil kitöltés

- **Bemenet:** Belépett felhasználó megnyitja a booking.html oldalt.
- **Elvárt eredmény:** A név és email mezők automatikusan tartalmazzák a profiladatokat.
- **Eredmény:** **Megfelelt**



4. Teszteset: Kötelező mezők validációja

- **Bemenet:** Foglalási kísérlet név vagy e-mail cím nélkül.
- **Elvárt eredmény:** A böngésző vagy a rendszer hibáüzenetet küld, a mentés nem történik meg.
- **Eredmény:** **Megfelelt**



14. Anatómia oldal

14.1 Funkcionális leírás

Az Anatómia modul célja, hogy interaktív módon mutassa be az emberi test felépítését a ForgeX felhasználói számára. A felület segítségével a látogatók mélyebb betekintést nyerhetnek az izomrendszerbe és a csontvázszerkezetbe, elősegítve ezzel a tudatosabb és biztonságosabb edzést.

14.1.1 Főbb funkciók:

- Interaktív 3D-s testmodell megjelenítése külső API (BioDigital) segítségével.
- Váltási lehetőség az izomzat (Muscular System) és a csontváz (Skeletal System) nézetei között.
- Teljes körű modellmanipuláció: forgatás, közelítés (zoom) és boncolás (dissect) funkciók az iFrame-en belül.
- Reszponzív navigációs fejléc és lábléc integráció.
- Többnyelvű támogatás a Google Translate modulon keresztül.

14.2 Frontend felépítés (HTML és CSS)

14.2.1 Szerkezeti felépítés (anatomy.html)

Az oldal három fő részre tagolódik. A fejléc után a tartalom egy szekció-headerrel kezdődik, amely vizuálisan elkülöníti a modult a divider-line és az egyedi vízszintes vonal (topLine) segítségével. A központi rész az anatomy-section, amely magában foglalja a vezérlőgombokat (selecterBtn) és a dinamikus tartalom befogadására szolgáló iframeContainer-t.

14.2.2 Megjelenés és reszponzivitás (anatomy.css, style.css)

- Vezérlők: A gombok (skeletonBtn3D, muscleBtn3D) fix méretűek (350x100px), és az aktuálisan kiválasztott állapotot a márka zöld színe (#00ca65) jelzi.
- Megjelenítő: Az iFrame konténere (frameSize) alapértelmezetten a képernyő szélességének 80%-át foglalja el (max. 800px), rögzített 800px-es magassággal, biztosítva a stabil vizuális élményt.
- Adaptivitás: A globális style.css alapelveit követve az oldal kisebb kijelzőkön is használható, a rögzített háttér (fixed-bg) pedig mélységet ad a tartalomnak.

14.3 Kliensoldali logika (JavaScript)

14.3.1 Felhasználói interakciók kezelése

Az anatomy.js felel a felhasználói gombnyomások feldolgozásáért. A DOMContentLoaded esemény lefutása után a szkript referenciákat készít a DOM elemekre, és eseményfigyelőket (Event Listeners) rendel a váltógombokhoz.

14.3.2 Dinamikus tartalomfrissítés

A rendszer nem tölti be egyszerre mindkét modellt a sávszélesség-optimalizálás érdekében. A váltás folyamata:

- A felhasználó rákattint az egyik gombra (pl. Csontvázszerkezet).
- A JS törli az iframeContainer aktuális tartalmát.
- Beilleszti a célzott iFrame HTML kódját, amely tartalmazza a specifikus BioDigital azonosítót (uaid).
- Frissíti a gombok vizuális állapotát az active CSS osztály hozzáadásával/eltávolításával.

14.3.3 iFrame konfiguráció

Az iFrame-ek beágyazása biztonságos sandbox környezetben történik. A paraméterezés során (URL query stringek) kikapcsolásra kerülnek a felesleges UI elemek (pl. ui-center=false), miközben engedélyezettek a tudományos vizsgálathoz szükséges eszközök (ui-dissect=true, ui-info=true).

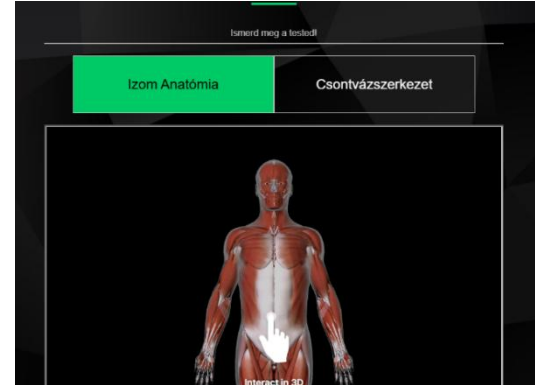
14.3.4 Hibakezelés

A szkript tartalmaz egy alapvető biztonsági ellenőrzést: csak akkor próbálja meg az eseményfigyelőket hozzárendelni, ha mindkét gomb és a konténer is létezik a DOM-ban. Ez megakadályozza a JavaScript hibák futását más oldalakon, ahol a globális script fájlok is be vannak töltve.

4. Tesztelési napló

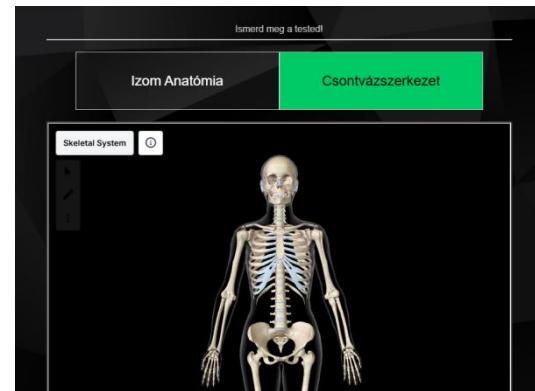
1. Teszteset: Alapértelmezett betöltés

- Bemenet: anatomy.html megnyitása.
- Elvárt eredmény: Az Izom Anatómia gomb aktív, az izomrendszer 3D-s modellje tölt be az iFrame-be.
- Eredmény: Megfelelt



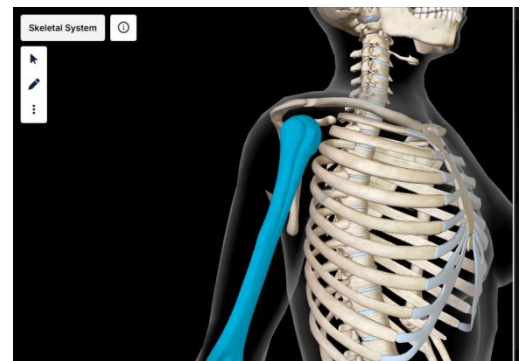
2. Teszteset: Nézet váltása

- Bemenet: Kattintás a "Csontvázszerkezet" gombra.
- Elvárt eredmény: Az Izom modell eltűnik, a Csontváz modell tölt be, a gomb színe zöldre vált.
- Eredmény: Megfelelt



3. Teszteset: iFrame funkcionalitás

- Bemenet: Egér görgetés (scroll) az iFrame felett.
- Elvárt eredmény: A 3D-s modell ráközelít (zoom) a testre.
- Eredmény: Megfelelt



4. Teszteset: Gyors váltás (Stresszteszt)

- Bemenet: A két gomb egymás utáni gyors kattintgatása.
- Elvárt eredmény: A rendszer nem fagy le, mindig az utoljára kattintott nézet iFrame kódja generálódik le.
- Eredmény: Megfelelt

15. BMI Kalkulátor

15.1 Funkcionális leírás

A BMI kalkulátor modul célja, hogy a ForgeX felhasználói gyorsan és vizuálisan szemléletes módon kapjanak visszajelzést az egészségi állapotukról a testtömeg-indexük alapján.

15.1.1 Főbb funkciók:

- **Számítás:** Testsúly (kg) és magasság (cm) alapján történő BMI kalkuláció egy tizedesjegy pontossággal.
- **Vizuális állapotjelző:** Egy SVG alapú, dinamikusán töltődő körív, amely a BMI érték függvényében változtatja a töltöttségét és a színét.
- **Színkódolt kiértékelés:** Automatikus kategóriába sorolás (sovány, normál, túlsúly, elhízás) a nemzetközi egészségügyi standardoknak megfelelően.
- **Szöveges tanácsadás:** Rövid, motiváló vagy figyelmeztető leírás megjelenítése az eredmény mellé.
- **Informatív segédlet:** Egy külön információs kártya, amely elmagyarázza a BMI jelentőségét és korlátait (pl. sportolók esetében).

15.2 Frontend felépítés (HTML és CSS)

15.2.1 Szerkezeti felépítés (calculatorBMI.html)

Az oldal struktúrája a `bmi-wrapper` elemen belül két fő kártyára (`card`) oszlik, amely egy modern, kétszlopos elrendezést biztosít:

Kalkulátor kártya (`calculator-card`): Ez a bal oldali fő egység. Tartalmazza az adatbeviteli űrlapot (`#bmiForm`) és az eredményjelző területet (`result-area`). Itt található az a speciális SVG grafika is, amely a köríves animációért felel.

Információs kártya (`info-card`): A jobb oldali egység, amely elméleti háttérrel biztosít. Tartalmazza a kategóriák listáját (`category-list`) színes indikátorokkal (`dot`), valamint egy kiemelt láblécet (`info-footer`) a fontos figyelmeztetéseknek.

15.2.2 Megjelenés és reszponzivitás (calculator.css)

- **Grafikai elemek:** Az eredményt megjelenítő kör (`progress-ring`) és a számok színe dinamikusan változik a JavaScript segítségével, de a stílusukat a CSS definiálja (pl. `cubic-bezier` animáció a sima töltődéshez).
- **Input design:** A beviteli mezők nagy méretűek, középre igazított szöveggel és egyedi fókusz-stílussal rendelkeznek a könnyű használat érdekében.
- **Reszponzivitás:** `@media` lekérdezések biztosítják, hogy 1024px alatti felbontásnál a két kártya egymás alá kerüljön, 600px alatt pedig az eredményjelző kör és a mellette lévő szöveg is függőlegesen rendeződjön, megőrizve az olvashatóságot telefonon is.

```
.progress-ring {  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 120px;  
  height: 120px;  
  transform: rotate(-90deg);  
}
```

15.3 Programozási logika (calculatorBmi.js)

15.3.1. Számítási algoritmus

Az űrlap elküldésekor a rendszer megakadályozza az alapértelmezett oldalfrissítést. A magasságot centiméterből méterbe váltja. Az eredményt egy tizedesjegyre kerekítve adja vissza, és a következő képlet alapján számítja ki az eredményt:

$$BMI = \frac{testsuly(kg)}{magassag(m)^2}$$

15.3.2 Dinamikus UI frissítés (**updateUI** függvény)

Ez a funkció felelős az oldal interaktivitásáért. A kapott BMI érték alapján egy logikai elágazásrendszer kiválasztja:

- A megfelelő **kategória nevet** (pl. "Normál").
- A hozzá tartozó **színt** (kék, zöld, sárga vagy piros).
- A rövid **tanácsot**.

15.3.3 SVG Animáció technológiája

A körív töltöttségét a `stroke-dashoffset` tulajdonság manipulálásával érjük el. A JavaScript kiszámítja a BMI arányát a 40-es határértékhez képest, majd a kör pontos kerülete alapján beállítja az eltolást. Ez hozza létre azt a látványos hatást, ahogy a kör a számítás gomb megnyomásakor "befut" a helyére.

A kör kerületének meghatározása

```
const circumference : number = 326.72;
```

A kitöltési arány kiszámítása. A rendszer a 40-es BMI értéket tekinti a 100%-os telítettségnek

```
const percentage : number = Math.min( values: bmi / 40, 1);
```

A `stroke-dashoffset` érték beállítása a kerületből levonjuk a kitöltendő részt, így kapjuk meg az eltolást.

```
const offset : number = circumference - (percentage * circumference);
```

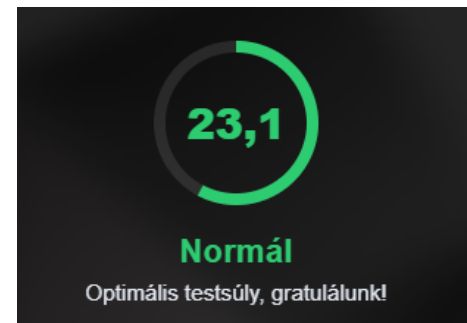
Vizuális alkalmazás és színváltás

```
bmiCircle.style.strokeDashoffset = offset;  
bmiCircle.style.stroke = color;
```

15.4 Tesztelési napló

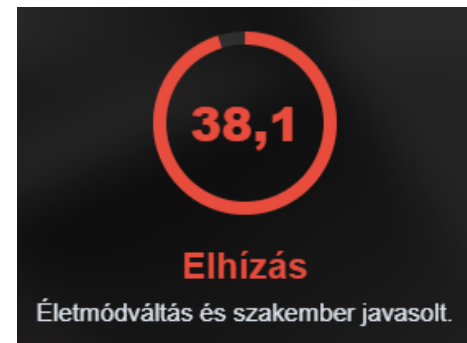
1. Teszteset: Normál BMI számítás

- **Bemenet:** 75 kg, 180 cm
- **Elvárt eredmény:** ~23,1 érték, zöld szín, "Normál" státusz, gratuláló leírás.
- **Eredmény:** Megfelelt



2. Teszteset: Színváltás és extrém értékek

- **Bemenet:** 110 kg, 170 cm
- **Elvárt eredmény:** >30 érték, piros szín, "Elhízás" státusz, figyelmeztető leírás.
- **Eredmény:** Megfelelt



3. Teszteset: Üres mezők kezelése

- **Bemenet:** Üresen hagyott súly mező
- **Elvárt eredmény:** A HTML5 `required` attribútuma miatt a böngésző figyelmeztetést dob, a JS nem fut le.
- **Eredmény:** Megfelelt



A form titled "TESTSÚLY (KG)" with a text input field. The input field contains a placeholder symbol. Below the input field, a white error message box with a red exclamation mark icon says: "Kérjük, töltsd ki ezt a mezőt." Below the error message, the number "180" is displayed.

16. Elérhetőségek

16.1 Funkcionális leírás

Az Elérhetőségek modul célja, hogy a ForgeX ügyfelei és partnerei számára többcsatornás kommunikációs lehetőséget biztosítson. Az oldal központi eleme egy intelligens kapcsolatfelvételi űrlap, amely lehetővé teszi a közvetlen üzenetküldést a megfelelő részlegnek, anélkül, hogy a felhasználónak el kellene hagynia a böngészőt.

16.1.1 Főbb funkciók:

- **Interaktív Kapcsolatfelvételi Űrlap:** EmailJS integrációval ellátott űrlap, amely azonnali e-mail továbbítást tesz lehetővé a kiválasztott tárgy (pl. Garancia, Rendelés) alapján.
- **Dinamikus Visszajelzés:** Egyedi modális ablakok tájékoztatják a felhasználót a küldés sikerességéről vagy esetleges hibákról.
- **Stilizált Google Maps integráció:** Egyedi CSS szűrőkkel ellátott térkép, amely illeszkedik az oldal sötét megjelenéséhez.
- **Közvetlen Elérhetőségek:** Kattintható telefonszámok és előre paraméterezett Gmail-hivatkozások a gyorsabb ügyintézés érdekében.
- **Strukturált Információközlés:** Logikailag elkülönített kártyák a telephelyi adatoknak, a nyitvatartásnak és az ügyfélszolgálati rendnek.

16.2 Frontend felépítés (HTML és CSS)

16.2.1 Szerkezeti felépítés (contacts.html)

Az oldal a `contact-page-content` konténeren belül egy rácsos elrendezést (Grid) használ a tartalom szervezésére:

Információs kártyák (`contact-info-grid`): Két fő oszlopra oszlik:

- **Helyszín kártya:** A fizikai címek és a raktár átvételi pontjainak részletezése.
- **Support kártya:** A telefonos és e-mailes elérhetőségek gyűjtőhelye, kiemelt színekkel a fontos adatoknál.

Űrlap konténer (`contact-form-container`): Egy teljes szélességű (grid-column: 1 / - 1) blokk, amely tartalmazza az összes beviteli mezőt és a küldő gombot.

Térkép szekció (`map-section`): Egy beágyazott `iframe`, amely a telephely pontos helyét mutatja meg.

16.2.2 Megjelenés és reszponzivitás (contacts.css)

A design itt is a **Glassmorphism** stílust követi, de kiegészül néhány egyedi megoldással:

- **Térkép szűrők:** Az `iframe` egy speciális CSS `filter`-t kapott (`invert`, `hue-rotate`), amely a Google Maps alapértelmezett színeit a ForgeX sötét/neon színeihez igazítja. Hover hatásra a térkép visszavált eredeti színeire.

```
.map-placeholder iframe {  
  width: 100%;  
  height: 100%;  
  border: none;  
  filter: grayscale(20%) invert(90%) hue-rotate(180deg)  
         brightness(95%) contrast(90%);  
}
```

- **Űrlap stílus:** Az input mezők és a **select** elem áttetsző fekete hátteret kaptak, fókusz esetén pedig neonzöld világító keretet.

```
@media (max-width: 768px) {
  .contact-info-grid {
    display: flex !important;
    flex-direction: column !important;
    grid-template-columns: 1fr !important;
    gap: 20px !important;
    width: 100% !important;
    padding: 0 !important;
  }
}
```

- **Reszponzív Grid:** 768px alatti felbontásnál a kétszlopos elrendezés egyoszloposra vált (**flex-direction: column**), így mobilon is kényelmesen kitölthető marad az űrlap.

```
document.body.appendChild(overlay);

const closeModal = () => {
  overlay.classList.remove('active');
  setTimeout(() => overlay.remove(), 300);
};

overlay.querySelector('button').addEventListener('click', closeModal);
overlay.addEventListener('click', (event) => {
  if (event.target === overlay) {
    closeModal();
  }
});

setTimeout(() => overlay.classList.add('active'), 10);
}
```

16.3 Programozási logika (contact-sender.js)

16.3.1 EmailJS Integráció

A rendszer nem igényel saját backend szerveret az üzenetek továbbításához. Az **emailjs.init** segítségével a kliensoldali JavaScript közvetlenül kommunikál az EmailJS szolgáltatással. A **sendForm** függvény összegyűjti az űrlap adatait és elküldi azokat a beállított e-mail sablon alapján.

```
(function () {
  emailjs.init("mrWqT0EGKMuyLPIYL");
})();
```

16.3.2 Állapotkezelés és UX

A küldés megnyomásakor a `submit-btn` inaktívvá válik (`disabled = true`), és a felirata "Küldés folyamatban..."-ra vált. Ez megakadályozza, hogy a felhasználó többször is elküldje ugyanazt az üzenetet türelmetlenségből.

```
const submitButton :HTMLInputElement = document.getElementById( elementId: 'submit-btn');
```

16.3.3 Egyedi Modális visszajelzés (showStatusModal)

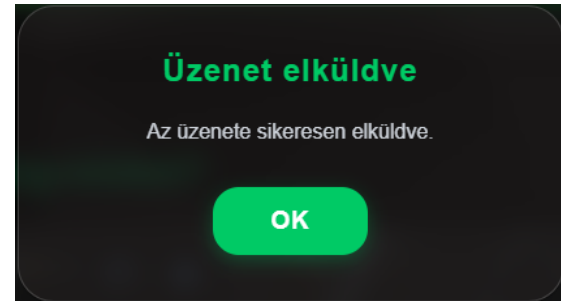
A program nem a böngésző egyszerű `alert()` ablakát használja, hanem egy dinamikusan létrehozott DOM elemet. Ez a modális ablak stílusában illeszkedik a weboldalhoz, és egy `setTimeout` segítségével sima áttűnéssel jelenik meg, professzionálisabb élményt nyújtva a hiba vagy siker esetén.

```
function showStatusModal(title, message) :void { Show usages  Balint67 *
  const oldOverlay :Element = document.querySelector( selectors: '.custom-modal-overlay');
  if (oldOverlay) oldOverlay.remove();
  const overlay :HTMLDivElement = document.createElement( tagName: 'div');
  overlay.className = 'custom-modal-overlay';
  overlay.innerHTML = `
    <div class="custom-modal-box">
      <h3>${title}</h3>
      <p>${message}</p>
      <div class="modal-buttons">
        <button class="modal-btn modal-btn-primary" type="button">OK</button>
      </div>
    </div>
  `;
};
```


16.4 Tesztelési napló

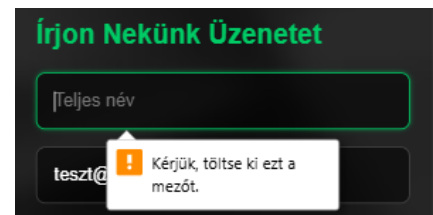
1. Teszteset: Sikeres üzenetküldés

- **Bemenet:** Minden mező érvényes kitöltése, tárgy választása.
- **Elvárt eredmény:** Az űrlap alaphelyzetbe áll (**reset**), és megjelenik a "Sikeres küldés" modális ablak.
- **Eredmény:** **Megfelelt**



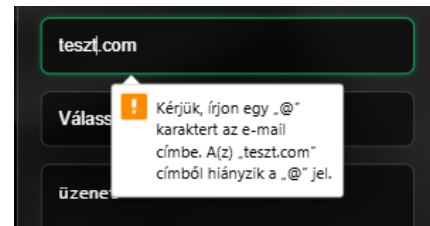
2. Teszteset: Kötelező mezők ellenőrzése

- **Bemenet:** Hiányos név vagy üres üzenetmező.
- **Elvárt eredmény:** A böngésző megállítja a küldést, és jelzi a hiányzó adatot a felhasználónak.
- **Eredmény:** **Megfelelt**



3. Teszteset: E-mail formátum validáció

- **Bemenet:** Hibás e-mail cím megadása.
- **Elvárt eredmény:** A rendszer jelzi, hogy a megadott e-mail cím formátuma érvénytelen.
- **Eredmény:** **Megfelelt**



4. Teszteset: Térkép hover effektus

- **Bemenet:** Egérkurzor mozgatása a térkép fölé.
- **Elvárt eredmény:** A sötétített, szűrt térkép visszavált színes, normál nézetre.
- **Eredmény:** **Megfelelt**



17. Vásárlói vélemények

17.1 Funkcionális leírás

A [reviews.html](#) oldal célja a társadalmi bizonyíték (social proof) szolgáltatása. A rendszer lehetővé teszi, hogy a felhasználók valós időben osszák meg tapasztalataikat, amelyek azonnal láthatóvá válnak minden látogató számára, függetlenül attól, hogy milyen eszközről böngészik az oldalt.

17.1.1 Főbb képességek:

- **Dinamikus tartalom:** A vélemények nem fixen kódoltak, hanem a felhőalapú Firebase Firestore adatbázisból töltődnek be minden megnyitáskor.
- **Valós idejű interakció:** A bejelentkezett felhasználók új véleményt hozhatnak létre, amely beküldés után azonnal, oldalfrissítés nélkül megjelenik a listában.
- **Saját tartalom kezelése:** A rendszer felismeri az aktuális felhasználót, és csak a saját maga által írt véleményeknél teszi lehetővé a törlést.
- **Értékelési rendszer:** Interaktív, 5 csillagos skála a vizuális visszajelzéshez.

17.2 Frontend felépítés és Megjelenés

18.2.1 Design alapelvek ([reviews.css](#))

- **Glassmorphism kártyák:** A vélemények áttetsző, elmosott háttérű dobozokban jelennek meg, amelyek kiemelik a szöveget a sötét háttér előtt.
- **Dinamikus Avatarok:** A rendszer a felhasználó nevéből generál kezdőbetűs ikonokat (pl. Kovács Bence -> KB), biztosítva a rendezett megjelenést akkor is, ha nincs profilkép.
- **Modal ablakok:** Az űrlap és a törlés megerősítése animált, felugró ablakokban történik.

17.2.2 Reszponzivitás

A vélemények elrendezése modern CSS Grid technológiát használ. Mobilon a kártyák automatikusan egymás alá rendeződnek, a funkciógombok pedig kényelmesen elérhető, nagyobb méretet vesznek fel.

17.3 Technikai megvalósítás (Backend & Logic)

17.3.1 Adatbázis integráció (review-handler.js)

Az oldal lelke a Firebase Firestore moduláris integrációja:

- **Aszinkron adatkezelés:** Az `onAuthStateChanged` figyeli a felhasználó státuszát, az `addDoc` és `getDocs` függvények pedig kezelik a felhőbe történő írást és olvasást.
- **Biztonságos törlés:** A törlés funkció előtt a rendszer egy egyedi Confirm Modal segítségével kér megerősítést, a tényleges törlést pedig a Firebase `deleteDoc` metódusa végzi.
- **Dátumformázás:** Egyedi segédfüggvény alakítja át a Firestore időbélyegzőit a magyar nyelvnek megfelelő `YYYY.MM.DD.` formátumra.

17.3.2 Logikai számítások és algoritmusok

A háttérben futó szkript több matematikai és string-manipulációs műveletet végez a helyes megjelenítés érdekében:

- **Avatar-generáló algoritmus:** A felhasználó által megadott teljes nevet szóközők mentén darabolja fel, majd minden részből kinyeri az első karaktert:

```
const getInitials = (name) => {  
  name  
    .trim()  
    .split( splitter: /\s+/ ) // A nevet szóközők mentén darabokra vágja  
    .map((part :string ) => part[0] || '') // Minden darabból kiveszi az első karaktert  
    .join('') // Összefűzi őket  
    .toUpperCase() // Nagybetűssé alakítja  
    .slice(0, 2); // Levágja, hogy maximum 2 karakter maradjon (pl. "Nagy János" -> "NJ")  
}
```

- **Csillag-arány számítás:** A rendszer egy ciklust futtat le 1-től 5-ig, és minden lépésben összehasonlítja az aktuális indexet a tárolt értékkel:

```
const createStars = (rating) :HTMLDivElement => {  
  const stars = document.createElement( tagName: 'div' );  
  stars.className = 'stars'; // Ezt formázod majd CSS-ben  
  
  for (let index = 1; index <= 5; index += 1) {  
    const icon = document.createElement( tagName: 'i' );  
    // Ha az index kisebb vagy egyenlő a kapott értékeléssel, akkor teli csillag (fas),  
    // egyébként üres csillag (far) lesz.  
    icon.className = index <= Number(rating) ? 'fas fa-star' : 'far fa-star';  
    stars.appendChild(icon);  
  }  
  
  return stars;  
};
```

17.3.3 Hibaüzenetek és Felhasználói Visszajelzések

A rendszer egyedi modális ablakokat és vizuális jelzéseket alkalmaz a hibák kezelésére:

- **Bejelentkezés hiánya:** Figyelmeztetés ("Sign in required") és átirányítás a [signIn.html](#)-re, ha vendégként próbálnak írni.

```
if (!name || !text) {
  await forgeXModal(
    title: 'Hiányzó információ',
    message: 'Kérjük, a vélemény elküldéséhez töltsön fel nevet és üzenetét.'
  );
  return;
}
```

- **Hiányos kitöltés:** Jelzés, ha a név vagy a szöveg üresen marad a küldés gomb megnyomásakor.

```
if (!currentUser) {
  await forgeXModal(
    title: 'Bejelentkezés szükséges',
    message: 'A vélemény írás művelethez bejelentkezés szükséges.'
  );
  window.location.href = 'signIn.html';
  return;
}
```

- **Gomb**

állapotváltozása: Beküldéskor a gomb felirata "Sending..."-re vált és inaktívvá válik, megakadályozva a duplikálást.

```
const originalButtonText = submitButton.innerText;
submitButton.disabled = true;
submitButton.innerText = 'Sending...';
```

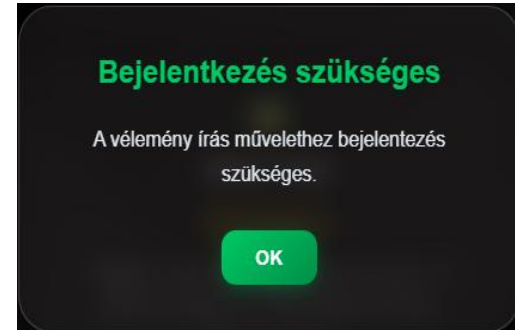
- **Törlés megerősítése:** Biztonsági párbeszédablak a véletlen adatvesztés elkerülésére.

```
deleteButton.addEventListener( type: 'click', listener: () :void => {
  pendingDeleteId = reviewId;
  openModal(deleteModal);
});
return deleteButton;
};
```

17.4 Tesztelési napló

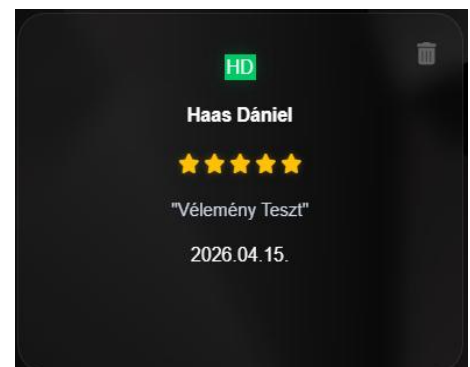
1. Teszteset: Jogosultságok és hozzáférés

- **Bemenet:** "Vélemény írása" gomb megnyomása kijelentkezett állapotban.
- **Elvárt eredmény:** Figyelmeztető modal megjelenése és átirányítás a [signIn.html](#)-re.
- **Eredmény:** **Megfelelt.**



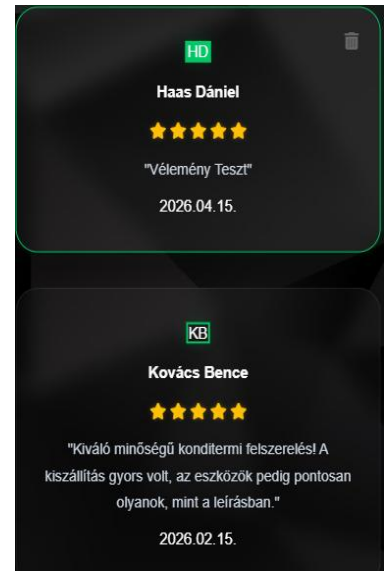
2. Teszteset: Új vélemény beküldése

- **Bemenet:** Név, 5 csillag és szöveg megadása, majd küldés.
- **Elvárt eredmény:** Az új vélemény azonnal megjelenik a lista élén, a beküldő neve és monogramja helyesen látszik.
- **Eredmény:** **Megfelelt.**



3. Teszteset: Törlési funkció védelme

- **Bemenet:** Más felhasználó által írt vélemény megtekintése.
- **Elvárt eredmény:** A kuka ikon nem jelenik meg, az idegen tartalom nem törölhető.
- **Eredmény:** Megfelelt.

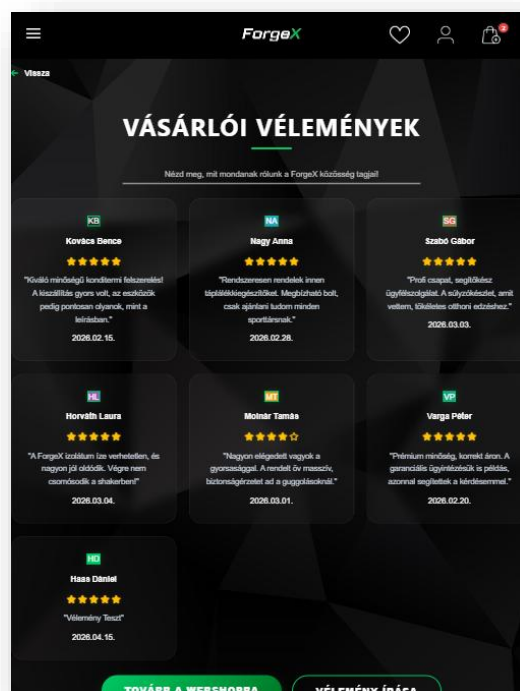
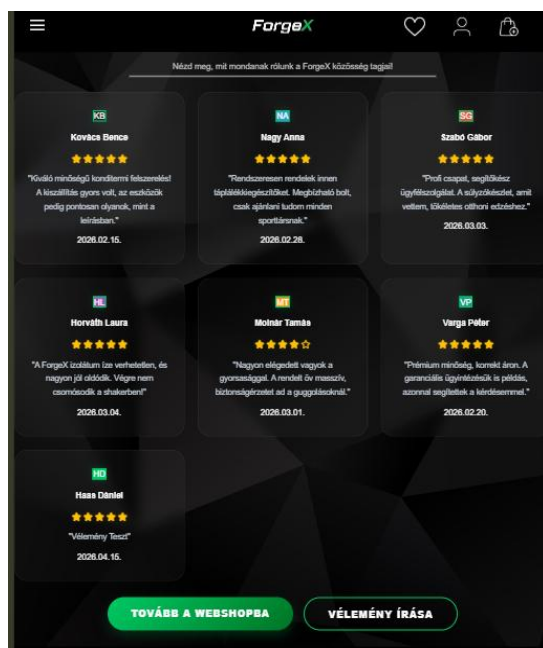


4. Teszteset: Adatbázis szinkron

- **Bemenet:** Oldal megnyitása két különböző böngészőből.
- **Elvárt eredmény:** Mindkét felületen ugyanazok a vélemények jelennek meg, azonos sorrendben.
- **Eredmény:** Megfelelt.

Dani számítógépéről készült fénykép

Bálint számítógépéről készült fénykép



18. Gyakran Ismételt Kérdések

18.1 Funkcionális leírás

A [questions.html](#) oldal célja az ügyfélszolgálati terhelés csökkentése és a vásárlói döntés megkönnyítése. A felületen a felhasználók választ kaphatnak a szállítással, garanciával, fizetéssel és az edzőtermek üzemeltetésével kapcsolatos leggyakoribb kérdésekre.

18.1.1 Főbb jellemzők:

- **Interaktív Harmonika (Accordion) rendszer:** A válaszok alapértelmezett állapotban rejtve vannak, így az oldal átlátható marad.
- **Kategorizált tartalom:** A kérdések lefedik a webshop (szállítás, garancia) és a fizikai edzőtermek (nyitvatartás, géppark) témaköreit is.
- **Gyors navigáció:** A felhasználók egy kattintással visszatérhetnek az előző oldalra vagy tovább léphetnek a Shop felületére.

18.2 Frontend felépítés és Megjelenés

18.2.1 Design alapelvek ([questions.css](#))

- **Glassmorphism kártyák:** A kérdések `backdrop-filter: blur(12px)` hatással rendelkező, áttetsző kártyákon jelennek meg.
- **Vizuális hierarchia:** A kérdések fehér színűek és félkövérek, míg a válaszok lágyabb, szürke (`#cbd5e1`) árnyalatot kaptak a jobb olvashatóság érdekében.
- **CTA integráció:** Az oldal alján egy feltűnő, zöld színátmenetes gomb ösztönzi a felhasználót a vásárlás folytatására.

18.2.2 Az animáció technikai megvalósítása

Az oldal egyik legfontosabb része a válaszok kinyílásának mechanizmusa. A hagyományos `display: none` helyett egy modern **CSS Grid trükköt** alkalmazunk a folyékony animációhoz:

- **A Wrapper logikája:** A `.faq-answer-wrapper` alaphelyzetben `grid-template-rows: 0fr`-rel rendelkezik.

```
.faq-answer-wrapper {  
  display: grid !important;  
  grid-template-rows: 0fr !important;  
  transition: grid-template-rows 0.4s cubic-bezier(0.4, 0, 0.2, 1) !important;  
}
```

- **A nyitás mechanizmusa:** Aktív állapotban (`.active`) ez az érték `1fr`-re változik. Mivel a `grid-template-rows` animálható tulajdonság, a böngésző simán "kihúzza" a tartalmat.

```
.faq-card.active .faq-answer-wrapper {  
  grid-template-rows: 1fr !important;  
}
```

- **Belső eltolás:** A válasz szövege (`p` tag) egy extra `translateY(-10px)` eltolást és `opacity: 0` értéket kap, ami a kinyíláskor egy felfelé úszó hatással válik láthatóvá.

```
.faq-answer p {  
  padding: 0 30px 25px 30px !important;  
  color: #cbd5e1 !important;  
  line-height: 1.8 !important;  
  margin: 0 !important;  
  opacity: 0;  
  transform: translateY(-10px);  
  transition: all 0.3s ease !important;  
}
```

18.3 Technikai megvalósítás (Backend & Logic)

18.3.1 JavaScript vezérlés (**questions.html** belső script)

Az állapotváltást egy egyszerű és gyors JavaScript függvény kezeli

- **Működés:** A függvény a fejléc (**faq-header**) kattintásakor megkeresi a szülő kártyát és hozzáadja/eltávolítja az **.active** osztályt. Ez indítja be a CSS-ben definiált grid-animációt és az ikon elforgatását.

18.3.2 Navigációs logika

- **Vissza gomb:** A **javascript:history.back()** metódust használja, ami javítja a UX-et, hiszen a felhasználó pontosan oda tér vissza (pl. egy termékoldalra), ahonnan a kérdései felmerültek.
- **Dinamikus SEO:** Az oldal tartalmazza a szükséges Meta tageket és reszponzív viewport beállításokat a keresőoptimalizálás érdekében.

18.4 Tesztelési napló

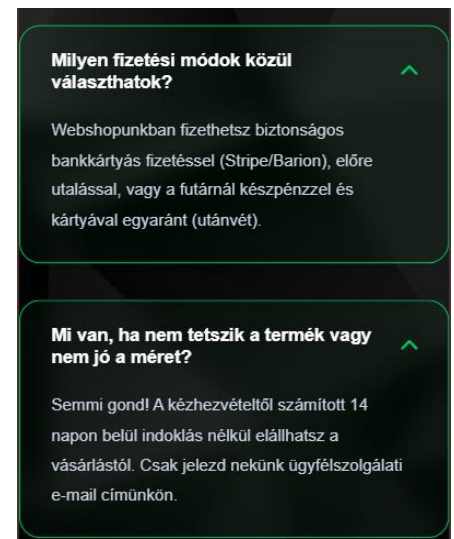
1. Teszteset: Harmonika működése

- **Bemenet:** Kattintás egy kérdésre.
- **Elvárt eredmény:** A válasz simán legördül, a nyíl ikon 180 fokkal elfordul, a kártya szegélye zöldre vált.
- **Eredmény:** **Megfelelt.**



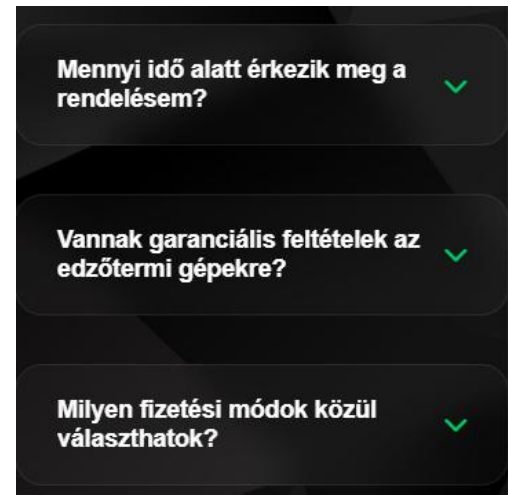
2. Teszteset: Többszörös nyitás

- **Bemenet:** Több kérdés egymás utáni megnyitása.
- **Elvárt eredmény:** Minden kártya megőrzi a saját állapotát, több válasz is lehet egyszerre nyitva.
- **Eredmény:** **Megfelelt.**



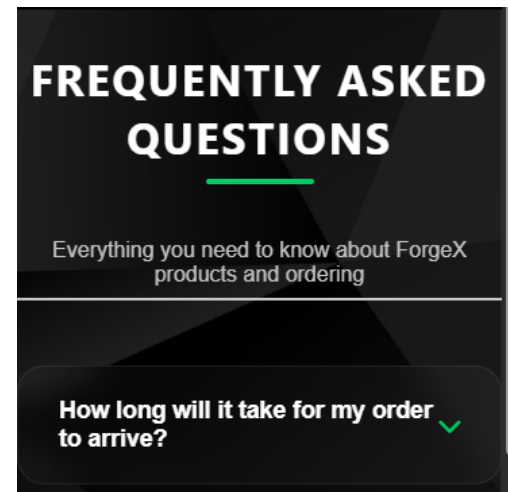
3. Teszteset: Reszponzivitás

- **Bemenet:** Nézet váltása 375px szélességre (mobil).
- **Elvárt eredmény:** A kártyák kitöltik a rendelkezésre álló szélességet, a szöveg olvasható marad, a paddingek megfelelően alkalmazkodnak.
- **Eredmény:** **Megfelelt.**



4. Teszteset: Fordító modul

- **Bemenet:** Nyelvváltás a fejlécben.
- **Elvárt eredmény:** A Google Translate modul megfelelően inicializálódik és lefordítja a statikus FAQ tartalmat is.
- **Eredmény:** **Megfelelt.**



19. Süti beállítások

19.1 Funkcionális leírás

A [cookies.html](#) oldal feladata, hogy átlátható és szabályozható módon tegye lehetővé a felhasználók számára a webhely által használt sütik kezelését.

19.1.1 Főbb képességek:

- **Granuláris szabályozás:** A látogató külön-külön dönthet az analitikai és marketing célú adatgyűjtésről.
- **Perzisztens adattárolás:** A beállítások a böngésző bezárása után is megmaradnak.
- **Interaktív visszajelzés:** Valós idejű vizuális megerősítés a beállítások mentésekor.

19.2 Frontend felépítés és Megjelenés

20.2.1 Design alapelvek ([cookies.css](#))

A kártya-alapú elrendezés segít az információk emészthető tálalásában:

- **Grid Layout:** A [.cookies-layout](#) egy [auto-fit](#) alapú rácsot használ, amely a képernyő szélességétől függően 1, 2 vagy 3 oszlopba rendezi a kategóriákat.

```
.cookies-layout {  
  display: grid !important;  
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)) !important;  
  gap: 25px !important;  
  width: 100% !important;  
  margin-top: 30px !important;  
}
```

- **Interaktív Kapcsolók (Toggle Switches):** A hagyományos jelölőnégyzetek helyett esztétikus, animált csúszkákat alkalmazunk.
 - **Állapotok:** Bekapcsolt állapotban a kapcsoló ForgeX-zöld ([#00ca65](#)) izzást és árnyékot kap.

- **Kártya Hover effekt:** A kártyák fölé víve az egérmutatót, azok 10 pixelt emelkednek (`translateY(-10px)`), és a szegélyük zölden világítani kezd.

```
.cookie-card:hover {
  transform: translateY(-10px) !important;
  background: rgba(255, 255, 255, 0.06) !important;
  border-color: #00ca65 !important;
  box-shadow: 0 15px 35px rgba(0, 0, 0, 0.4), 0 0 15px rgba(0, 202, 101, 0.1) !important;
}
```

19.2.2 Vizuális jelzések

- **Status Tags:** Az "Alapvető sütik" kategória nem kikapcsolható, ezt egy speciális `.mandatory` osztályú címke jelzi, vizuálisan megkülönböztetve a választható opcióktól.

```
.status-tag.mandatory {
  background: rgba(0, 202, 101, 0.1) !important;
  border: 1px solid rgba(0, 202, 101, 0.3) !important;
  color: #00ca65 !important;
  padding: 10px 20px !important;
  border-radius: 50px !important;
  font-weight: 700 !important;
  font-size: 0.85rem !important;
  text-transform: uppercase !important;
}
```

19.3. Technikai megvalósítás (Backend & Logic)

19.3.1 Adatkezelés (cookies.js)

A rendszer nem adatbázist, hanem a kliensoldali **LocalStorage**-t használja a preferenciák tárolására.

- **Betöltési logika:** Az oldal megnyitásakor a `localStorage.getItem` lekéri a korábbi döntéseket. Ha nincs mentett adat (első látogatás), az analitika alapértelmezetten aktív.

```
const savedAnalytics :string = localStorage.getItem( key: 'forgex_analytics');
const savedMarketing :string = localStorage.getItem( key: 'forgex_marketing');
```

- **Mentési folyamat:** A gombra kattintva a `localStorage.setItem` rögzíti a kapcsolók aktuális `checked` állapotát (true/false).

```
saveButton.addEventListener( type: 'click', listener: () :void => {
  localStorage.setItem('forgex_analytics', analyticsBox.checked);
  localStorage.setItem('forgex_marketing', marketingBox.checked);
}
```

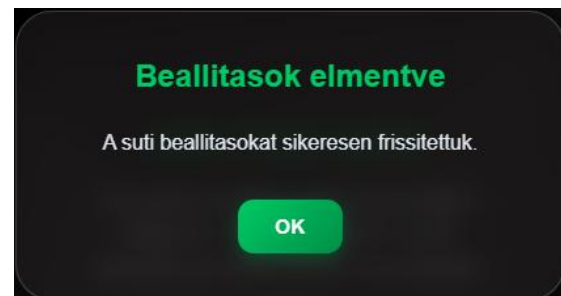
19.3.2 UI Logika és Animációk

- **Állapotjelző gomb:** A mentés gomb szövege a kattintás után ideiglenesen *"BEALLITASOK MENTVE!"*-re vált, majd visszaugrik az eredetire, miközben egy fényerő-növekedési effektet kap.
- **Egyedi Modális Ablak:** A `showStatusModal` függvény dinamikusan hoz létre egy értesítést a DOM-ban.
 - **Zárási mechanizmus:** Az ablak nemcsak az "OK" gombra, hanem a sötétített háttérre kattintva is bezáródik, biztosítva a sima kilépést (`setTimeout` az animáció lefutásához).

19.4 Tesztelési napló

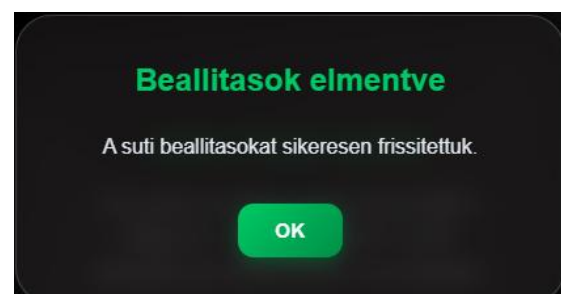
1. Teszteset: Alapértelmezett állapot

- **Bemenet:** Első oldalmegnyitás.
- **Elvárt eredmény:** Alapvető és Analitikai sütik aktívak, Marketing inaktív.
- **Eredmény:** *Megfelelt.*



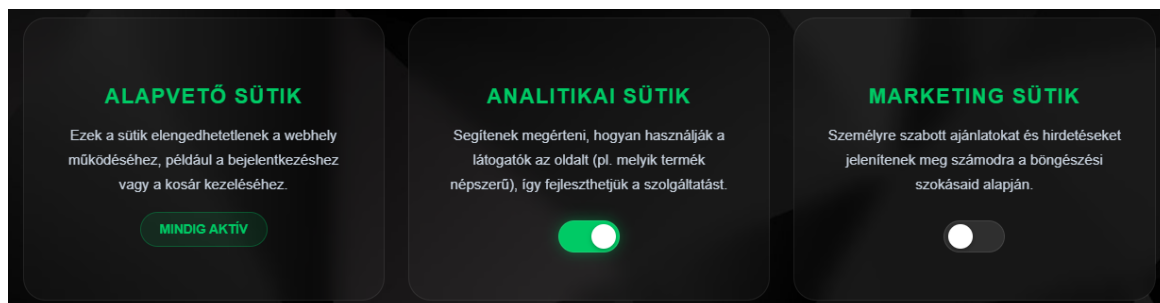
2. Teszteset: Beállítások mentése

- **Bemenet:** Marketing süti bekapcsolása, majd mentés.
- **Elvárt eredmény:** Megjelenik a sikerességet jelző modális ablak, a LocalStorage-ben a `forge_x_marketing` értéke `true` lesz.
- **Eredmény:** *Megfelelt.*



3. Teszteset: Adatperzisztencia

- **Bemenet:** Böngésző frissítése (F5) a mentés után.
- **Elvárt eredmény:** A kapcsolók a frissítés után is a mentett pozícióban maradnak.
- **Eredmény:** **Megfelelt.**



20 Közösségi média integráció és marketing szerepe

A ForgeX digitális jelenlétének erősítése érdekében a láblécben (footer) elhelyezésre kerültek a projekt közösségi média csatornáinak hivatkozásai.

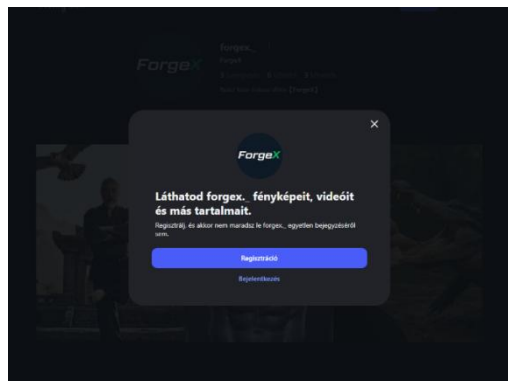
20.1 Alkalmazott megoldások:

- **Platformok:** Instagram, Facebook és TikTok direkt elérés.
- **Technikai megvalósítás:** A hivatkozások minden esetben új böngészőlapon nyílnak meg (`target="_blank"`), hogy a felhasználó ne hagyja el véglegesen a weboldalt. A biztonság érdekében a hivatkozások tartalmazzák a releváns attribútumokat.
- **Vizuális megjelenítés:** Az ikonok és a feliratok egységes stílust kaptak. A CSS-ben definiált `animation-bottom` osztály segítségével hover állapotban finom elmozdulással (`transform: translateY(-2px)`) és színváltással reagálnak a felhasználói interakcióra.
- **Márkakonzisztencia:** Az SVG alapú vagy optimalizált PNG logók (pl. `instaLogo.png`, `facebookLogo.png`) segítik a vizuális azonosítást, miközben illeszkednek a sötét témájú arculatba.

20.2 Tesztelési napló (Közösségi média kiegészítés)

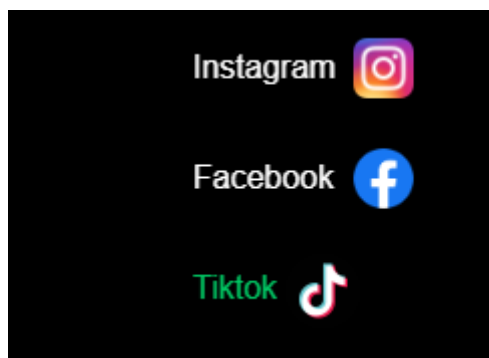
1 Teszteset: Külső hivatkozások működése

- **Bemenet:** Kattintás az Instagram logóra a láblécben.
- **Elvárt eredmény:** Az Instagram profil egy új böngészőlapon sikeresen betöltődik, miközben a ForgeX oldala is nyitva marad az eredeti lapon.
- **Eredmény:** Megfelelt



2 .Teszteset: Közösségi média ikonok interakciója

- **Bemenet:** Az egér kurzorának mozgatása a TikTok hivatkozás fölé.
- **Elvárt eredmény:** A szöveg színe átvált a ForgeX zöld színére (#00ca65), és az elem finoman megemelkedik.
- **Eredmény:** Megfelelt



21 Adatbázis

21.1 firebase.js – Infrastruktúra és konfiguráció

Ez a modul a Firebase szolgáltatások központi inicializálását végzi. Feladata, hogy a projekt összes többi modula egységesen ugyanazt a hitelesítési és adatbázis-kapcsolatot használja.

Az inicializálás egy változó vagy objektum kezdeti értékének beállítása.

21.1.1 A modul főbb feladatai:

- A Firebase alkalmazás inicializálása a projekt konfigurációs adataival
- Az Authentication szolgáltatás példányosítása
- A Firestore adatbázis példányosítása
- Az auth és db objektumok exportálása a többi JavaScript modul számára

A megoldás előnye, hogy a hitelesítéssel és adatbázissal dolgozó fájlok nem külön-külön hozzák létre saját kapcsolataikat, hanem ugyanazt a központi példányt használják. Ez átláthatóbb és karbantarthatóbb szerkezetet eredményez.

21.1.2 A modulhoz kapcsolódó fontosabb adatbázis-gyűjtemények:

- users: felhasználói adatok
- carts: kosár elemei
- favorites: kedvencek
- bookings: foglalások
- Technikai részletek:
- auth-utils.js – Hitelesítési segédfüggvények

Ez a modul a hitelesítési folyamathoz kapcsolódó közös segédfüggvényeket tartalmazza. Célja, hogy a regisztrációs, bejelentkezési és profilkezelési logika ne ismétlődjön több fájlban.

A modul főbb feladatai:

- Annak eldöntése, hogy szükséges-e e-mail megerősítés
- A megerősítő e-mail elküldéséhez szükséges beállítások összeállítása
- Megerősítő e-mail kiküldése
- Az e-mail megerősítési állapot Firestore-ba történő szinkronizálása
- Az aktuális felhasználói állapot frissítése

A modul használatának előnye, hogy a hitelesítési folyamat egységes marad a különböző oldalak között, és az e-mail megerősítési logika központilag kezelhető.

- Függőségek: A `firebase-app.js`, `firebase-auth.js` és `firebase-firestore.js` importálása biztosítja a szükséges funkciókat.
- Eseménykezelés: Mivel a fájl csak az inicializálást végzi, nincsenek benne DOM-manipulációs logikák, ami tisztán tartja a függőségi gráfot.

```
export function requiresEmailVerification(user) { Show usages  Balint67
  if (!user) return false;

  const usesPasswordProvider :boolean = (user.providerData || []).some(
    (provider :T) :boolean => provider.providerId === PASSWORD_PROVIDER_ID
  );

  return usesPasswordProvider && !user.emailVerified;
}
```

```
// =====...
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js";

// =====
// ♦ FIREBASE CONFIGURATION
// =====
const firebaseConfig :{...} = {
  apiKey: "AIzaSyA-qcUAb6bgjwxfttKBHJOcCe0Jw4u6HME",
  authDomain: "forgex-2026.firebaseio.com",
  projectId: "forgex-2026",
  storageBucket: "forgex-2026.firebaseioapp",
  messagingSenderId: "73260668042",
  appId: "1:73260668042:web:4b29784cde814c3a628973"
};

// =====
// ♦ FIREBASE INITIALIZATION
// =====
const app = initializeApp(firebaseConfig);

// =====|
// ♦ EXPORTED FIREBASE SERVICES
// =====
export const auth = getAuth(app); Show usages  Mészáros Bálint
export const db = getFirestore(app); Show usages  Mészáros Bálint
```

21.2 utils.js – Segédfüggvények és UI Komponensek

A `utils.js` egy központi tárolóhely az alkalmazásban gyakran használt segédfüggvények számára. Jelenleg egyetlen, de kulcsfontosságú elemet tartalmaz: a `forgeXModal` függvényt. Ez a modul felel azért, hogy egységes, a webshop arculatához illeszkedő vizuális visszajelzést (modális ablakot) nyújtson a felhasználónak, kiváltva a böngésző alapértelmezett, nem testreszabható `alert()` vagy `confirm()` dobozait.

21.2.1 A modul főbb jellemzői:

1. Dinamikus DOM-manipuláció: A függvény nem a HTML-ben előre megírt elemeket használja, hanem a hívás pillanatában hozza létre őket a `document.createElement` segítségével. Ez a "tisztá" módszer: amíg nincs szükség rá, nem terheli a DOM-ot, majd a művelet végeztével el is távolítja az elemeket (`overlayElement.remove()`).
2. Promise-alapú aszinkron működés: A függvény egy `Promise`-t (ígéretet) ad vissza. Ez azért zseniális, mert lehetővé teszi, hogy a kód "megvárja", amíg a felhasználó rákattint az OK vagy Cancel gombra.
 - *Példa:* `const confirmed = await forgeXModal(...)` Ez az `async/await` minta modern, olvasható és szinkron logikát tesz lehetővé egy alapvetően aszinkron felhasználói interakciónál.
3. Testreszabhatóság: A függvény paraméterei (`title`, `message`, `isConfirm`) lehetővé teszik, hogy ugyanazt a logikát használj egy egyszerű figyelmeztetésre (Alert) és egy döntést igénylő párbeszédpanelre (Confirm) is.

21.2.2 Kódmagyarázat:

- HTML Injektálás: A template literálok (`${...}`) használatával a dinamikus tartalom könnyen beilleszthető a modal dobozba.
- Animáció kezelés: A `setTimeout` használata biztosítja a CSS `active` osztály hozzáadását a létrehozás utáni pillanatban, így a böngésző tudja animálni a megjelenést (fade-in hatás).
- Tisztítás (Cleanup): A `resolve` meghívása után a `setTimeout` gondoskodik a modal eltávolításáról, ami memóriakezelési szempontból is példaértékű.

```
document.body.appendChild(overlayElement);

setTimeout( handler: () :void => {
  overlayElement.classList.add('active');
}, timeout: 10);

const closeModal = (result) :void => { Show usages  Balint67 +1
  overlayElement.classList.remove( tokens: 'active');

  setTimeout( handler: () :void => {
    overlayElement.remove();
    resolve(result);
  }, timeout: 300);
};
```


21.3 auth-header.js – Dinamikus Navigáció és Állapotkezelés

Ez a fájl egy ún. Observer (megfigyelő) mintát valósít meg, amely figyeli a felhasználói autentikációs állapotot, és ennek megfelelően módosítja a weboldal navigációs elemeit (pl. a fejléc vagy az alsó navigációs sáv linkjeit). Ez a fájl teszi lehetővé hogy a profil oldalt csak a bejelentkezett felhasználók láthassák és amíg nincs bejelentkezve addig az ember ikon a jobb felső sarokban a bejelentkezés oldalra viszi a felhasználót de amint megtörtént a bejelentkezés ugyanez a gomb már a profil oldalra irányít ahol látni lehet a felhasználó adatait valamint a foglalásokat a különféle órákra.

21.3.1 A modul főbb feladatai:

1. Állapotfigyelés (`onAuthStateChanged`): A fájl a Firebase `onAuthStateChanged` metódusát használja, amely egy "figyelő" (listener). Ez a függvény azonnal lefut, amint a felhasználó bejelentkezik vagy kijelentkezik. Így az alkalmazás képes valós időben reagálni a státuszváltásra anélkül, hogy az oldálnak újra be kellene töltenie.
2. DOM Szinkronizáció: A `querySelectorAll` segítségével a kód megkeres minden olyan linket (`signIn.html` vagy `#profile-link`), amely a beléptetéssel kapcsolatos. Amint az autentikációs állapot megváltozik, a `link.href` tulajdonságát átírja:
 - Bejelentkezett állapotban: A hivatkozások a `profil.html`-re mutatnak.
 - Kijelentkezett állapotban: A hivatkozások a `signIn.html`-re mutatnak.
3. Egységes élmény: Mivel a kód egyszerre kezeli a fejléctet és az alsó navigációs menüt, a felhasználó bárhol is kattint az alkalmazásban, mindig a státuszának megfelelő oldalra jut.

21.3.2 Kódmagyarázat:

- **DOMContentLoaded:** A kód biztosra megy, és csak azután fut le, miután a HTML struktúra már betöltődött a böngészőbe. Ez megelőzi a "null pointer" hibákat (amikor a JavaScript megpróbál manipulálni egy elemet, ami még nem is létezik).
- Rugalmasság: A **querySelectorAll** használata különösen professzionális, mert így a kód akkor is működni fog, ha a jövőben újabb navigációs elemeket adunk az oldalhoz

```
document.addEventListener( type: 'DOMContentLoaded', listener: () :void => {  Balint67 +1

  // Elements selection
  const profileLinks :NodeListOf<Element> = document.querySelectorAll( selectors: 'a[href="signIn.html"], #profile-link');
  const cartBadge :HTMLElement = document.getElementById( elementId: 'cart-count');
  let cartUnsubscribe :null = null;
  let isStorageListenerAttached :boolean = false;

  /**
   * Updates the cart badge UI visibility and value
   * @param {number} total - The total number of items in the cart
   */
  const updateBadgeUI = (total :number ) :void => { Show usages  Balint67
    if (!cartBadge) return;

    if (total > 0) {
      cartBadge.innerText = total;
      cartBadge.style.display = 'flex'; // Show badge if cart is not empty
    } else {
      cartBadge.style.display = 'none'; // Hide badge if cart is empty
    }
  };
};
```

– a JS automatikusan megtalálja és felruházza őket a dinamikus linkelési logikával.

21.4 Tesztelési napló

firebase.js A Firebase app, auth és A Firebase **Megfelelt**
inicializálás db objektumok sikeresen szolgáltatások
inicializálódnak, nincs elérhetőek a
konzolhiba. konzolban.

utils.js Promise Az OK/Mégse gombra **async/await** hívások **Megfelelt**
működés kattintva a Promise a kódban helyesen
resolve (lefut) és az ablak várnak az
bezárul.resolve (lefut) és eredményre.
az ablak bezárul. helyesen várnak az
eredményre.

utils.js DOM Az ablak bezárása után a Nincs visszamaradt **Megfelelt**
tisztítás **div** eltűnik a DOM-ból. elem a dokumentum
végén.

utils.js Modális A **forgeXModal** Overlay megjelenik **Megfelelt**
megjelenés hívásakor a DOM-ba az oldalon a
bekerül az overlay és a megfelelő
tartalom. tartalommal.

utils.js DOM Az ablak bezárása után a Nincs visszamaradt **Megfelelt**
tisztítás div eltűnik a DOM-ból. elem a dokumentum
végén.

auth-header.js Kijelentkezett állapotban A linkek átirányítása **Megfelelt**
kijelentkezés a linkek signIn.html-re kijelentkezett
mutatnak. állapotban helyes.

auth-header.js Bejelentkezett állapotban A linkek átirányítása **Megfelelt**
bejelentkezés a linkek profil.html-re bejelentkezett
mutatnak. állapotban helyes.

